CENTRO UNIVERSITÁRIO IBMR

ELIAS BORGES VIANA

ARTHUR SILVA DE CERQUEIRA

PLATAFORMA DE TELEMETRIA OBD-II SIMULADA: PROJETO E VALIDAÇÃO EM SOFTWARE-IN-THE-LOOP

Rio de Janeiro

2026

ELIAS BORGES VIANA

ARTHUR SILVA DE CERQUEIRA

PLATAFORMA DE TELEMETRIA OBD-II SIMULADA: PROJETO E VALIDAÇÃO EM SOFTWARE-IN-THE-LOOP

Trabalho de Conclusão de Curso apresentado ao curso de Engenharia da Computação do Centro Universitário IBMR como requisito parcial para obtenção do título de Bacharel em Engenharia da Computação.

Orientador: Prof. FRANCISCO JOSÉ RODRIGUES DA SILVA JUNIOR

Rio de Janeiro

2026

ELIAS BORGES VIANA

ARTHUR SILVA DE CERQUEIRA

PLATAFORMA DE TELEMETRIA OBD-II SIMULADA: PROJETO E VALIDAÇÃO EM SOFTWARE-IN-THE-LOOP

Este Trabalho de Conclusão de Curso foi julgado adequado à obtenção do título de Bacharel em Engenharia da Computação e aprovado em sua forma final pelo Curso de Engenharia da Computação do Centro Universitário IBMR.

Rio de Janeiro, ____ de _____ de 2026.

BANCA EXAMINADORA:

_____

Orientador: Prof. Francisco José Rodrigues da Silva Junior

Centro Universitário IBMR

_____

Prof. Nome do Professor, Dr./Ms./Bel./Lic.

Centro Universitário IBMR

_____

Prof. Nome do Professor, Dr./Ms./Bel./Lic.

Centro Universitário IBMR

Às nossas famílias, pelo apoio e pela paciência ao longo de toda a graduação.

AGRADECIMENTOS

Agradecemos às nossas famílias, pelo apoio constante durante o curso. Ao professor Francisco José Rodrigues da Silva Junior, pela orientação deste trabalho. Aos professores e colegas do curso de Engenharia da Computação do Centro Universitário IBMR, pelo aprendizado compartilhado ao longo da graduação.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

( ) Não utilizei ferramentas de Inteligência Artificial na elaboração deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial como apoio nas seguintes etapas:

Finalidade do uso: foi utilizado o assistente Claude (Anthropic) na redação do texto dos capítulos, na organização das referências, no desenvolvimento do código do simulador, do coletor, do servidor, do dashboard e do painel de controle, na elaboração dos scripts de ensaio e dos gráficos e na execução dos testes e dos ensaios em ambiente computacional. Os autores foram responsáveis pela definição do tema, dos objetivos e do escopo, pela execução do ensaio com o *broker* Mosquitto e da validação com aplicativos de terceiros em computador próprio, e pela leitura e revisão do conteúdo final.

Declaro que a análise crítica, reflexiva e a autoria intelectual deste trabalho são de minha responsabilidade.

RESUMO

O acesso aos dados que um veículo produz depende, em grande parte, de ferramentas de diagnóstico de custo elevado e arquitetura fechada, e desenvolver software de telemetria exige um veículo, um adaptador e condições repetíveis que nem sempre existem. Além disso, em um veículo real o valor verdadeiro de cada grandeza é desconhecido, de modo que a exatidão da leitura só pode ser afirmada, não medida. Este trabalho apresenta uma plataforma aberta de telemetria OBD-II construída inteiramente em software, na qual um veículo simulado, uma unidade de controle virtual conforme a ISO 15765-4 (com ISO-TP e os modos 01, 02, 03, 04, 07 e 09), um coletor, um servidor com banco SQLite, um dashboard web e um painel de controle do simulador reproduzem a cadeia completa de coleta, armazenamento e visualização. O coletor depende apenas de uma interface de transporte, o que permite substituir o simulador por hardware sem alterá-lo, e uma camada de compatibilidade ELM327 por TCP permite que bibliotecas e aplicativos de terceiros funcionem como clientes. A pesquisa é aplicada e exploratória, por desenvolvimento experimental, com método de software-in-the-loop e sementes fixas. Foram executados 405 testes automatizados, com cobertura de 89% e de 97% a 100% nos módulos de protocolo, e sete ensaios de desempenho em uma única máquina. Em 19.070 leituras, nenhuma apresentou erro de decodificação acima de metade da resolução; o ciclo de dez parâmetros teve percentil 95 de 115,6 ms, a latência do coletor ao banco teve percentil 95 de 3,5 ms e não houve perdas. Os ensaios mostraram que o ciclo cresce linearmente com o atraso da ECU e com o número de parâmetros e que a perda de quadros reduz a vazão sem perda de dados no servidor; com um intermediário Mosquitto real no mesmo computador, o MQTT teve latência média de 0,6 ms contra 3,0 ms do HTTP. A camada ELM327 foi validada com a biblioteca python-OBD e com o aplicativo OBD Auto Doctor. Os resultados valem para o simulador e não afirmam nada sobre veículos reais; a substituição do simulador por hardware e o ensaio de campo ficam como segunda etapa.

Palavras-chave: Telemetria veicular. OBD-II. Simulação. *Software-in-the-loop*. ELM327.

ABSTRACT

Access to the data a vehicle produces still depends largely on expensive, closed diagnostic tools, and developing telemetry software requires a vehicle, an adapter and repeatable conditions that are not always available. Moreover, on a real vehicle the true value of each quantity is unknown, so the accuracy of a reading can only be asserted, not measured. This work presents an open OBD-II telemetry platform built entirely in software, in which a simulated vehicle, a virtual control unit compliant with ISO 15765-4 (with ISO-TP and services 01, 02, 03, 04, 07 and 09), a collector, a server with an SQLite database, a web dashboard and a simulator control panel reproduce the whole chain of acquisition, storage and visualization. The collector depends only on a transport interface, so the simulator can be replaced by hardware without changing it, and an ELM327 compatibility layer over TCP lets third-party libraries and apps work as clients. The research is applied and exploratory, based on experimental development, using a software-in-the-loop method with fixed seeds. A total of 405 automated tests were run, with 89% coverage overall and 97% to 100% in the protocol modules, along with seven performance experiments on a single machine. In 19,070 readings, none showed a decoding error above half of the resolution; the ten-parameter cycle had a 95th percentile of 115.6 ms, the collector-to-database latency had a 95th percentile of 3.5 ms and there were no losses. The experiments showed that the cycle grows linearly with the ECU delay and with the number of parameters, and that frame loss reduces throughput without losing data on the server; with a real Mosquitto broker on the same computer, MQTT had a mean latency of 0.6 ms against 3.0 ms for HTTP. The ELM327 layer was validated with the python-OBD library and with the OBD Auto Doctor app. The results hold for the simulator and say nothing about real vehicles; replacing the simulator with hardware and a field test remain as a second stage.

Keywords: Vehicle telemetry. OBD-II. Simulation. Software-in-the-loop. ELM327.

LISTA DE FIGURAS

Figura 1: Consulta de rotação (modo 01, PID 0C) sobre CAN.....30
Figura 2: Leitura do VIN por ISO-TP (primeiro quadro, controle de fluxo e dois quadros consecutivos).....31
Figura 3: Arquitetura da plataforma.....37
Figura 4: Módulos e sentido das dependências.....38
Figura 5: Ciclo de aquisição do coletor.....45
Figura 6: Modelo entidade-relacionamento do banco SQLite (colunas principais)….48
Figura 7: Dashboard de telemetria com rotação sobreposta em 3000 rpm e o código P0300 armazenado.....49
Figura 8: Painel de controle do simulador com monitor de quadros.....49
Figura 9: Implantação com Docker Compose.....52
Figura 10: Valor real e valor decodificado nos 10 PIDs (primeiros 300 ciclos do E1).....57
Figura 11: Tempo de ciclo em função do atraso da ECU (E2).....58
Figura 12: Taxa por PID e leituras por segundo em função do número de PIDs (E3).....60
Figura 13: Timeouts e leituras por segundo em função da perda de quadros (E4).....61
Figura 14: Tempo de remontagem do VIN em função do STmin (E5).....62
Figura 15: Latência do coletor ao banco por canal de publicação (E7, Mosquitto real).....64

LISTA DE TABELAS

Tabela 1: Comparação entre trabalhos centrados em hardware e esta plataforma..... 26
Tabela 2: Modos de serviço OBD-II implementados..... 27
Tabela 3: PIDs implementados, fórmulas e resoluções..... 28
Tabela 4: Tipos de quadro do ISO-TP..... 30
Tabela 5: Requisitos funcionais (RF) e critérios de aceitação..... 35
Tabela 6: Requisitos não funcionais (RNF) e metas..... 35
Tabela 7: Módulos, responsabilidades e tamanho do código-fonte..... 38
Tabela 8: Portas e serviços..... 38
Tabela 9: Escolhas de tecnologia, alternativas consideradas e motivos..... 39
Tabela 10: Variáveis do modelo de veículo..... 41
Tabela 11: Catálogo de falhas do simulador..... 43
Tabela 12: Implementações da interface Transport..... 44
Tabela 13: Rotas do servidor da plataforma..... 47
Tabela 14: Pirâmide de testes..... 50
Tabela 15: Desenho dos ensaios..... 50
Tabela 16: Tarefas do desenvolvimento da primeira etapa..... 52
Tabela 17: Cobertura de testes nos módulos de protocolo..... 54
Tabela 18: Resultados do ensaio E1 (240 s, 10 PIDs)..... 55
Tabela 19: Erro de decodificação por PID no ensaio E1..... 56
Tabela 20: Requisitos não funcionais no ensaio E1..... 58
Tabela 21: Resultados do ensaio E2..... 58
Tabela 22: Resultados do ensaio E3..... 59
Tabela 23: Resultados do ensaio E4..... 60
Tabela 24: Tempo de remontagem do VIN (E5, 100 repetições)..... 61
Tabela 25: Resultados do ensaio E7 (60 s por canal, Mosquitto real)..... 64
Tabela 26: Atendimento aos requisitos..... 66
Tabela 27: Origem dos números do Capítulo 5..... 77
Tabela 28: Sessão ELM327 capturada contra o simulador..... 81
Tabela 29: Decisões de projeto selecionadas..... 82

LISTA DE ABREVIATURAS E SIGLAS

Sigla	Significado
ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface (Interface de Programação de Aplicações)
ASGI	Asynchronous Server Gateway Interface
BS	Block Size (tamanho do bloco, no ISO-TP)
CAN	Controller Area Network (Rede de Área do Controlador)
CF	Consecutive Frame (quadro consecutivo, no ISO-TP)
CRC	Cyclic Redundancy Check (Verificação Cíclica de Redundância)
CSV	Comma-Separated Values
DLC	Data Length Code (Código de Comprimento de Dados)
DTC	Diagnostic Trouble Code (Código de Falha de Diagnóstico)
ECU	Electronic Control Unit (Unidade Eletrônica de Controle)
FC	Flow Control (controle de fluxo, no ISO-TP)
FF	First Frame (primeiro quadro, no ISO-TP)
HIL	Hardware-in-the-Loop (hardware no laço)
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things (Internet das Coisas)
ISO	International Organization for Standardization
ISO-TP	Protocolo de transporte para CAN (ISO 15765-2)
JSON	JavaScript Object Notation
MAP	Manifold Absolute Pressure (Pressão Absoluta do Coletor)
MIL	Malfunction Indicator Lamp (Lâmpada Indicadora de Mau Funcionamento)
MQTT	Message Queuing Telemetry Transport
OBD-II	On-Board Diagnostics, segunda geração
P2	Tempo máximo de resposta da ECU a uma requisição (50 ms)
PCI	Protocol Control Information (informação de controle de protocolo)
PID	Parameter Identifier (Identificador de Parâmetro)
QoS	Quality of Service (Qualidade de Serviço)
RF	Requisito funcional
RNF	Requisito não funcional
RPM	Rotações por minuto
RTT	Round-Trip Time (Tempo de Ida e Volta)
SAE	Society of Automotive Engineers
SF	Single Frame (quadro único, no ISO-TP)
SIL	Software-in-the-Loop (software no laço)
SQL	Structured Query Language
STFT	Short-Term Fuel Trim (Ajuste de Combustível de Curto Prazo)
STmin	Separation Time minimum (tempo mínimo de separação entre quadros)

Sigla	Significado
TCP	Transmission Control Protocol
TWAI	Two-Wire Automotive Interface
VIN	Vehicle Identification Number (Número de Identificação do Veículo)

SUMÁRIO

1 INTRODUÇÃO.....16
1.1 CONTEXTO.....16
1.2 PROBLEMA E MOTIVAÇÃO.....17
1.3 SOLUÇÃO PROPOSTA.....18
1.4 OBJETIVOS.....19
1.4.1 Objetivo geral.....19
1.4.2 Objetivos específicos.....19
1.5 DELIMITAÇÃO DO ESCOPO.....20
1.6 METODOLOGIA.....20
1.7 ESTRUTURA DO TRABALHO.....21
2 REVISÃO BIBLIOGRÁFICA.....22
2.1 DA MANUTENÇÃO MECÂNICA AO DIAGNÓSTICO ELETRÔNICO.....22
2.2 REDES INTRAVEICULARES E O PROTOCOLO CAN.....23
2.3 TELEMETRIA VEICULAR E INTERNET DAS COISAS.....23
2.4 PROTOCOLOS DE MENSAGERIA.....24
2.5 SEGURANÇA DE SISTEMAS AUTOMOTIVOS.....24
2.6 SIMULAÇÃO NO DESENVOLVIMENTO AUTOMOTIVO.....25
2.7 TRABALHOS RELACIONADOS.....25
2.8 SÍNTESE.....26
3 FUNDAMENTAÇÃO TÉCNICA.....27
3.1 DIAGNÓSTICO DE BORDO OBD-II.....27
3.1.1 Modos de serviço.....27
3.1.2 Identificadores de parâmetros (PIDs).....28
3.1.3 Códigos de falha (DTC).....29
3.2 REDE CAN.....29
3.3 TRANSPORTE ISO-TP.....30
3.4 ADAPTADOR ELM327.....31
3.5 TECNOLOGIAS DE SOFTWARE.....32
3.5.1 Python e programação assíncrona.....32
3.5.2 python-can.....32
3.5.3 FastAPI, Uvicorn e Pydantic.....33
3.5.4 HTTP, WebSocket e MQTT.....33
3.5.5 SQLite e SQLAlchemy.....33
3.5.6 Front-end com HTML, JavaScript e SVG.....33

3.5.7 Docker e Docker Compose....33
3.5.8 Testes: pytest e hypothesis....34
3.6 SÍNTESE....34
4 MATERIAIS E MÉTODOS....35
4.1 REQUISITOS....35
4.2 ARQUITETURA....36
4.2.1 Módulos e dependências....37
4.2.2 Portas e processos....38
4.3 ESCOLHAS DE TECNOLOGIA....39
4.3.1 Por que simulador, e não apenas uma gravação de dados reais....39
4.3.2 Por que um transporte plugável....40
4.3.3 Por que o ELM327 como camada de compatibilidade....40
4.4 NÚCLEO DE PROTOCOLO....40
4.5 SIMULADOR....41
4.5.1 Modelo de veículo....41
4.5.2 ECU virtual....42
4.5.3 Injeção de falhas....43
4.6 TRANSPORTE E CAMADA ELM327....43
4.6.1 Interface `Transport`....
4.6.2 Implementações....44
4.6.3 Servidor ELM327....44
4.7 COLETOR....45
4.7.1 Publicadores....46
4.8 SERVIDOR, BANCO DE DADOS E INTERFACES....46
4.8.1 API do servidor....46
4.8.2 Esquema do banco de dados....47
4.8.3 Dashboard....48
4.8.4 Painel de controle do simulador....49
4.9 ESTRATÉGIA DE TESTES....49
4.10 ENSAIOS....50
4.11 AMBIENTE DE EXECUÇÃO E DOCKER....52
4.12 PROCESSO DE DESENVOLVIMENTO....52
5 RESULTADOS E DISCUSSÃO....54
5.1 TESTES AUTOMATIZADOS E COBERTURA....54
5.2 ENSAIO E1: EXATIDÃO E DESEMPENHO BASE....55
5.2.1 Exatidão da decodificação....55

5.2.2 Detecção da falha.....57
5.2.3 Requisitos RNF01 a RNF04.....57
5.3 ENSAIO E2: SENSIBILIDADE AO ATRASO DA ECU.....58
5.4 ENSAIO E3: NÚMERO DE PIDS.....59
5.5 ENSAIO E4: PERDA DE QUADROS.....60
5.6 ENSAIO E5: REMONTAGEM ISO-TP.....61
5.7 VALIDAÇÃO COM CLIENTES ELM327 DE TERCEIROS.....62
5.7.1 E6: cliente de soquete cru.....62
5.7.2 python-OBD.....63
5.7.3 OBD Auto Doctor.....63
5.8 ENSAIO E7: HTTP CONTRA MQTT.....64
5.9 REPRODUTIBILIDADE E AMEAÇAS À VALIDADE.....65
5.10 ATENDIMENTO AOS REQUISITOS.....66
5.11 DISCUSSÃO.....67
6 CONSIDERAÇÕES FINAIS.....68
6.1 CONCLUSÕES.....68
6.2 CONTRIBUIÇÕES.....69
6.3 LIMITAÇÕES.....69
6.4 TRABALHOS FUTUROS.....70
6.4.1 Segunda etapa: hardware.....70
6.4.2 Melhorias da plataforma.....71
6.5 CONSIDERAÇÃO FINAL.....72
REFERÊNCIAS.....73
APÊNDICE A: ORIGEM DOS NÚMEROS.....77
APÊNDICE B: COMO REPRODUZIR.....79
Instalação e testes.....79
Execução da plataforma.....79
Execução dos ensaios.....79
Repetição em outra máquina.....79
Validação com cliente real.....80
APÊNDICE C: SESSÃO ELM327 COM O SIMULADOR.....81
APÊNDICE D: DECISÕES DE PROJETO REGISTRADAS.....82

1 INTRODUÇÃO

Um automóvel moderno é, em boa medida, uma rede de computadores sobre rodas. Dezenas de unidades eletrônicas de controle (ECU, do inglês *Electronic Control Unit*) medem, calculam e decidem a todo instante: quanto combustível injetar, quando gerar a centelha, se a mistura está rica ou pobre, se a temperatura do motor está dentro da faixa esperada (DENTON, 2017; GUIMARÃES, 2007). Todas essas grandezas existem, em formato digital, dentro do veículo. O que varia é o quanto o proprietário, o mecânico independente, o estudante ou o pesquisador conseguem enxergar delas.

A porta de entrada padronizada para essas informações é o diagnóstico de bordo de segunda geração, o OBD-II (*On-Board Diagnostics*). Ele define um conector, um conjunto de serviços e uma lista de parâmetros que qualquer ferramenta compatível pode requisitar à ECU do motor (SAE INTERNATIONAL, 2016a; 2017). No Brasil, a Resolução CONAMA nº 354/2004 trata dos requisitos para a adoção de sistemas OBD nos veículos leves, com o objetivo de preservar a funcionalidade dos sistemas de controle de emissões (BRASIL, 2004). Na prática, isso significa que existe uma interface comum, mas o que se constrói em cima dela (leitura, armazenamento, visualização, análise) fica por conta de quem a usa.

Este trabalho trata dessa camada superior. Ele apresenta uma plataforma de telemetria OBD-II construída inteiramente em software: um veículo simulado, uma ECU virtual que responde conforme a norma, um coletor que requisita e decodifica os parâmetros, um servidor que armazena e distribui os dados, e um painel web que os exibe em tempo real. Nenhum componente físico é necessário para executá-la, e todos os resultados reportados foram obtidos por esse caminho.

1.1 CONTEXTO

A telemetria veicular reúne a coleta, a transmissão e a análise remota de dados de um veículo em operação. Ela sustenta aplicações como gestão de frotas, manutenção preditiva, seguros baseados em uso e diagnóstico remoto (COPPOLA; MORISIO, 2016; MALEKIAN et al., 2017). Em paralelo, a Internet das Coisas (IoT, do inglês *Internet of Things*) tornou comum a ideia de que objetos físicos publicam medições por uma rede, e que serviços em nuvem ou locais as consomem (ATZORI;

IERA; MORABITO, 2010; GUBBI et al., 2013; AL-FUQAHA et al., 2015). O veículo é um caso particularmente rico dessa ideia, porque já traz sensores, redes internas e uma interface de diagnóstico normatizada.

Por outro lado, o acesso aos dados veiculares é historicamente concentrado. Ferramentas de diagnóstico profissionais têm custo elevado e arquitetura fechada, e debates regulatórios sobre o chamado “direito de reparar” mostram que a disponibilidade de informação técnica é uma questão econômica e política, além de técnica (FEDERAL TRADE COMMISSION, 2021; UNIÃO EUROPEIA, 2018). Adaptadores baratos baseados no chip ELM327 popularizaram a leitura de dados por aplicativos de celular, mas o que se faz com os dados depois da leitura continua sendo, quase sempre, um problema de cada projeto.

Há ainda um aspecto de método. Quem desenvolve software de telemetria precisa testá-lo. Testar contra um veículo real traz limitações práticas: nem todo estudante tem acesso a um veículo com a interface adequada, o ensaio depende das condições do momento (temperatura, estado do motor, carga elétrica), e provocar uma falha de propósito para verificar se o sistema a detecta é, no mínimo, imprudente. Além disso, mesmo quando a leitura funciona, falta uma referência: o software informa que a rotação é 2000 rpm, mas qual era a rotação verdadeira? Sem essa resposta, não há como medir o erro do próprio sistema.

1.2 PROBLEMA E MOTIVAÇÃO

A dor que motiva este trabalho pode ser resumida em quatro pontos.

1. **Dependência de hardware para desenvolver e testar.** O ciclo “alterar o código, ligar o adaptador, conectar ao veículo, observar” é lento, caro e nem sempre possível. Isso afasta estudantes e pequenos laboratórios do tema.
2. **Ausência de verdade de referência.** Em um veículo real, o valor físico exato de cada grandeza em cada instante é desconhecido para quem mede. Sem ele, a exatidão da decodificação só pode ser afirmada, não medida.
3. **Falhas difíceis de reproduzir.** Códigos de falha, perda de mensagens, respostas atrasadas e quadros corrompidos são exatamente as situações que um sistema de telemetria precisa tratar bem, e são as mais difíceis de provocar de forma controlada e repetível em um carro.

4. **Resultados não repetíveis.** Ensaios com hardware e veículo reais dificilmente podem ser repetidos por outra pessoa nas mesmas condições, o que enfraquece a verificação independente dos resultados.

A motivação, portanto, é construir um ambiente em que o protocolo OBD-II, a camada de transporte, o coletor, o servidor e a interface possam ser desenvolvidos e medidos sem hardware, com valores de referência conhecidos, falhas injetáveis por comando e execuções repetíveis a partir de sementes numéricas fixas.

1.3 SOLUÇÃO PROPOSTA

A solução é uma plataforma de telemetria OBD-II simulada, aberta e executável com um comando (`make run` ou `docker compose up`). Ela reproduz a cadeia completa em cinco blocos:

- **Simulador.** Um modelo de veículo em passos de 10 ms gera velocidade, marcha, rotação, borboleta, carga, pressão do coletor, temperaturas, nível de combustível e tensão, seguindo perfis de condução. Sobre ele, uma ECU virtual responde às requisições conforme a ISO 15765-4, com ISO-TP completo e os modos 01, 02, 03, 04, 07 e 09.
- **Transporte plugável.** O coletor conhece apenas a interface `Transport`. Há implementações para laço local (*loopback*), barramento CAN virtual (`python-can`) e adaptador ELM327 por TCP.
- **Coletor.** Descobre os PIDs suportados, monta um plano de leitura com PIDs rápidos e lentos, valida cada resposta, decodifica pelas fórmulas da SAE J1979, mede o tempo de ida e volta e publica amostras numeradas.
- **Servidor.** Uma aplicação FastAPI recebe as amostras por HTTP ou MQTT, grava em SQLite, detecta perdas e duplicações pelo número de sequência e retransmite por WebSocket.
- **Interfaces.** Um painel web mostra a telemetria em tempo real e um segundo painel controla o simulador (valores, perfis, falhas, atraso e perda de quadros), com um monitor de quadros em hexadecimal.

Como o simulador conhece o valor físico verdadeiro de cada grandeza, o banco de dados guarda, ao lado de cada leitura decodificada, o valor real (`true_value`). Isso permite calcular o erro de decodificação por uma simples consulta

SQL. Como toda a aleatoriedade vem de sementes, qualquer pessoa pode repetir um ensaio e comparar os resultados.

Uma camada de compatibilidade ELM327 por TCP, na porta 35000, faz o simulador se comportar como um adaptador Wi-Fi. Com isso, bibliotecas e aplicativos de terceiros, escritos sem nenhum conhecimento deste projeto, conseguem conectar e ler dados, o que funciona como validação externa do formato das respostas.

1.4 OBJETIVOS

1.4.1 Objetivo geral

Projetar, implementar e validar, por testes automatizados e ensaios de desempenho, uma plataforma aberta de telemetria OBD-II independente de hardware, em que a cadeia completa de coleta, armazenamento e visualização possa ser desenvolvida, medida e repetida em ambiente de *software-in-the-loop*.

1.4.2 Objetivos específicos

1. Especificar os requisitos funcionais e não funcionais da plataforma e os critérios de aceitação de cada um.
2. Implementar o protocolo OBD-II sobre CAN (ISO 15765-4) como núcleo puro, sem entrada e saída, com ISO-TP, modos de serviço, PIDs e códigos de falha.
3. Implementar um simulador de veículo e uma ECU virtual determinísticos, com injeção de falhas de aplicação e de comunicação.
4. Implementar o coletor, o servidor, a persistência, o dashboard e o painel de controle do simulador.
5. Disponibilizar o transporte como interface plugável, de modo que um adaptador real possa substituir o simulador sem alterar o coletor.
6. Implementar a camada ELM327 por TCP e validá-la contra clientes de terceiros.
7. Medir exatidão de decodificação, tempo de ciclo, latência, perdas e sensibilidade a atraso, perda de quadros, tamanho do ciclo, espaçamento ISO-TP e canal de publicação (HTTP e MQTT).

8. Documentar as escolhas de tecnologia, as decisões tomadas e as limitações, de forma que o trabalho possa ser continuado em uma segunda etapa com hardware.

1.5 DELIMITAÇÃO DO ESCOPO

Este trabalho corresponde à **primeira etapa** de um projeto maior. A primeira etapa é inteiramente em software. A segunda etapa, fora deste texto, prevê a substituição do simulador por hardware (um microcontrolador emulando um adaptador ELM327 por Wi-Fi, ligado à rede CAN de um veículo) e um ensaio de campo. Ficam, portanto, fora do escopo deste trabalho:

- qualquer construção ou ensaio com hardware embarcado e qualquer medição em veículo real;
- a camada física do CAN (níveis elétricos, terminação, confirmação de recebimento entre nós) e a taxa de 500 kbit/s como grandeza medida;
- os protocolos K-Line, J1850 e CAN de 29 bits;
- comandos que atuem sobre o veículo: a plataforma é somente de leitura;
- Bluetooth, TLS e aplicativo móvel.

Por esse motivo, o trabalho não afirma nada sobre o comportamento de um veículo real. As conclusões valem para o simulador e para a plataforma que o cerca, e o termo adequado para o método é *software-in-the-loop*: o software sob teste (coletor e servidor) interage com um modelo em software do restante do sistema.

1.6 METODOLOGIA

Quanto à natureza, a pesquisa é aplicada, pois busca produzir uma solução para um problema prático. Quanto aos objetivos, é exploratória e descritiva, e quanto aos procedimentos, é um desenvolvimento experimental, com construção de um artefato e medição de seu desempenho em ensaios controlados (GIL, 2017; PRODANOV; FREITAS, 2013).

O desenvolvimento seguiu um fluxo em quatro passos, descritos em detalhe no Capítulo 4:

1. **Especificação.** Os requisitos (RF01 a RF12 e RNF01 a RNF08) e as interfaces entre módulos foram escritos antes do código, em documentos versionados no repositório.

2. **Implementação orientada por testes.** O trabalho foi dividido em tarefas pequenas, cada uma com seus testes escritos primeiro, e integrada por pedido de alteração (*pull request*) em um repositório Git.
3. **Ensaios.** Sete ensaios (E1 a E7) medem exatidão, desempenho e robustez; cada um grava configuração, sementes, máquina e resultados em arquivos CSV e JSON.
4. **Validação externa.** A camada ELM327 foi exercitada por bibliotecas e aplicativos de terceiros (python-OBD e OBD Auto Doctor).

Todas as medições deste texto vêm de um único computador, descrito na Seção 4.10. Não há segunda máquina medida, e o trabalho declara essa limitação em vez de supor reprodutibilidade entre máquinas. A repetição por outra pessoa é possível e está documentada (Apêndice B), mas não foi executada.

1.7 ESTRUTURA DO TRABALHO

O Capítulo 2 revisa a literatura sobre diagnóstico eletrônico, redes intraveiculares, telemetria e IoT, mensageria, segurança e simulação. O Capítulo 3 apresenta a fundamentação técnica: OBD-II, CAN, ISO-TP, ELM327 e as tecnologias de software empregadas. O Capítulo 4 descreve os materiais e métodos: requisitos, arquitetura, cada módulo, as escolhas de tecnologia com suas alternativas, a estratégia de testes e o desenho dos ensaios. O Capítulo 5 apresenta e discute os resultados. O Capítulo 6 reúne as considerações finais, as limitações e os trabalhos futuros, incluindo a segunda etapa com hardware.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo situa o trabalho na literatura. Ele percorre a evolução do diagnóstico veicular, as redes intraveiculares, a telemetria e a Internet das Coisas, os protocolos de mensageria, a segurança de sistemas automotivos e o papel da simulação no desenvolvimento, e termina com os trabalhos relacionados e a lacuna que esta plataforma pretende preencher.

2.1 DA MANUTENÇÃO MECÂNICA AO DIAGNÓSTICO ELETRÔNICO

Durante décadas, o diagnóstico de um motor dependeu da experiência do mecânico e de instrumentos simples: o som do motor, a cor da fumaça, a leitura de um manômetro. A introdução da injeção eletrônica de combustível e do controle eletrônico de ignição mudou esse quadro, porque o funcionamento do motor passou a ser decidido por programas que leem sensores e comandam atuadores (DENTON, 2017; GUIMARÃES, 2007). Esses programas conhecem, a cada instante, o estado do motor com uma precisão que nenhuma inspeção externa alcança.

A pressão regulatória sobre emissões acelerou a padronização. Para verificar que os sistemas de controle de emissões continuam funcionando durante toda a vida do veículo, as autoridades passaram a exigir que o próprio veículo se autodiagnostique e sinalize falhas ao condutor, por meio da lâmpada indicadora de mau funcionamento (MIL, do inglês *Malfunction Indicator Lamp*). O OBD-II é a segunda geração desse requisito, com conector, modos de serviço e códigos de falha padronizados (SAE INTERNATIONAL, 2016a; 2016b; 2017). No Brasil, a Resolução CONAMA nº 354/2004 estabelece os requisitos para a adoção desses sistemas nos veículos automotores leves (BRASIL, 2004), e na União Europeia a homologação de veículos também prevê o acesso a informações de diagnóstico e reparo (UNIÃO EUROPEIA, 2018).

O resultado é um ecossistema de ferramentas. Há *scanners* profissionais, de custo elevado e software proprietário, e há adaptadores de baixo custo, em geral baseados no chip ELM327, que se conectam por Bluetooth ou Wi-Fi a aplicativos de celular. A discussão sobre o direito de reparar mostra que o acesso a esses dados tem peso econômico: restrições de acesso e de informação técnica podem limitar o reparo independente (FEDERAL TRADE COMMISSION, 2021). Para um projeto

acadêmico, essa realidade se traduz em uma oportunidade: o padrão é aberto o bastante para que uma implementação independente seja possível, e é por isso que este trabalho a constrói.

2.2 REDES INTRAVEICULARES E O PROTOCOLO CAN

O número de ECUs por veículo cresceu de forma tão acentuada que ligar cada sensor a cada controlador por um fio dedicado tornou-se inviável. A solução foi usar barramentos digitais compartilhados. Navet et al. (2005) e Zeng, Khalid e Chowdhury (2016) descrevem esse panorama, em que coexistem várias tecnologias, cada uma com seu custo e sua finalidade (LIN para funções simples, CAN para controle, FlexRay para aplicações críticas).

O CAN (*Controller Area Network*) foi proposto pela Bosch e especificado na versão 2.0 em 1991 (BOSCH, 1991), e posteriormente padronizado pela ISO 11898 (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2015; 2016a). Suas características mais relevantes para este trabalho são: comunicação por difusão, em que toda mensagem chega a todos os nós; arbitragem não destrutiva por identificador, em que o identificador de menor valor vence a disputa pelo barramento; quadros curtos, com até 8 bytes de dados no CAN clássico; e detecção de erros por CRC e outros mecanismos. Johansson, Törngren e Nielsen (2005) discutem o uso do CAN em aplicações veiculares, e Davis et al. (2007) tratam da análise de escalonabilidade de mensagens CAN, ou seja, de como garantir que mensagens cumpram seus prazos em um barramento compartilhado.

Um ponto importante para a comparação com este trabalho é que o OBD-II usa o CAN como meio, mas não é o CAN. O OBD-II define o que se pergunta (modos e PIDs) e como se responde (formatos de mensagem), enquanto o CAN define como os bytes viajam. Entre os dois fica a camada de transporte ISO-TP, que permite enviar mensagens maiores que 8 bytes sobre quadros de 8 bytes. Esse conjunto é o que a ISO 15765-4 especifica para sistemas relacionados a emissões (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2021).

2.3 TELEMETRIA VEICULAR E INTERNET DAS COISAS

A Internet das Coisas descreve um cenário em que objetos com sensores e capacidade de comunicação publicam dados por redes IP. Atzori, Iera e Morabito

(2010) organizam a área em torno de três visões (orientada a “coisas”, a internet e a semântica), Gubbi et al. (2013) propõem uma arquitetura de referência centrada em nuvem e Al-Fuqaha et al. (2015) levantam tecnologias habilitadoras e protocolos, incluindo os de camada de aplicação.

No setor automotivo, o termo “carro conectado” abrange serviços que vão do infoentretenimento à gestão de frotas e ao diagnóstico remoto (COPPOLA; MORISIO, 2016). Uma arquitetura típica de telemetria tem quatro estágios: aquisição (leitura dos dados na rede interna do veículo), transmissão (envio por rede sem fio ou celular), armazenamento (banco local ou em nuvem) e visualização ou análise (painéis, alertas, aprendizado de máquina). A plataforma deste trabalho implementa os quatro estágios, com a aquisição sobre uma ECU simulada.

2.4 PROTOCOLOS DE MENSAGERIA

A escolha do protocolo que leva a amostra do coletor ao servidor é um problema clássico de IoT. O HTTP é universal e simples de depurar, mas é orientado a requisição e resposta e acrescenta cabeçalhos a cada envio. O MQTT (*Message Queuing Telemetry Transport*) adota o modelo de publicação e assinatura com um intermediário (*broker*), tem cabeçalhos mínimos e define três níveis de qualidade de serviço (QoS 0, 1 e 2) para escolher entre custo e garantia de entrega (OASIS, 2014). Hunkeler, Truong e Stanford-Clark (2008) apresentam a variante MQTT-S para redes de sensores sem fio, e Naik (2017) compara MQTT, CoAP, AMQP e HTTP em sistemas IoT, destacando que a escolha depende de requisitos de latência, consumo e confiabilidade. O Mosquitto, usado neste trabalho como *broker*, é uma implementação de código aberto do MQTT (LIGHT, 2017).

A plataforma deste trabalho suporta os dois canais, HTTP e MQTT, justamente para poder compará-los com as mesmas condições de coleta (ensaio E7). O WebSocket, padronizado pela RFC 6455 (FETTE; MELNIKOV, 2011), é usado em outro trecho: do servidor ao navegador, para o dashboard receber dados sem consultar repetidamente.

2.5 SEGURANÇA DE SISTEMAS AUTOMOTIVOS

A abertura das redes veiculares tem um lado de risco. Koscher et al. (2010) e Checkoway et al. (2011) mostraram, em análises experimentais, que interfaces

externas (inclusive a de diagnóstico) podem ser vetores de ataque a funções do veículo, e Smith (2016) sistematiza técnicas de análise de segurança automotiva. Dessa literatura decorre uma decisão de projeto assumida neste trabalho: a plataforma é **somente de leitura**. O modo 04 (limpeza de códigos de falha) existe apenas no simulador, para exercitar o protocolo, e a API de controle do simulador escuta, por padrão, apenas em `127.0.0.1`. Segurança de transporte (TLS) e autenticação ficam como trabalho futuro.

2.6 SIMULAÇÃO NO DESENVOLVIMENTO AUTOMOTIVO

A simulação é prática consolidada no desenvolvimento automotivo. Fathy et al. (2006) revisam a simulação com hardware no laço (HIL, do inglês *hardware-in-the-loop*), em que um controlador físico real é ligado a um modelo simulado da planta. Em contraste, no *software-in-the-loop* (SIL), tanto o software sob teste quanto o modelo do restante do sistema rodam em software, o que dispensa qualquer hardware, permite execução mais rápida ou determinística e facilita a automação de testes. Este trabalho se enquadra no segundo caso: o coletor e o servidor são o software sob teste, e a ECU e o veículo são modelos.

O preço do SIL é a distância em relação à realidade. Um modelo simplificado não reproduz toda a complexidade de um veículo, e o barramento virtual não tem a temporização do CAN físico. Por isso este trabalho evita extrapolações: os resultados descrevem a plataforma e o simulador, e não um veículo. A decisão de manter a interface `Transport` estável cumpre o papel inverso, o de deixar pronto o ponto em que o hardware real entrará.

2.7 TRABALHOS RELACIONADOS

Malekian et al. (2017) descrevem um sistema de gestão de frotas com OBD-II sem fio, em que um módulo conectado ao veículo transmite dados a um servidor. Rodríguez Rodríguez, Vento Álvarez e Inouye Rodríguez (2018) implementam uma ferramenta de diagnóstico OBD-II sobre CAN com Arduino, com foco na leitura e interpretação de parâmetros. Silva (2025) desenvolve, em trabalho de conclusão de curso, um sistema embarcado para telemetria veicular. Esses trabalhos concentram o esforço no hardware e na integração com o veículo, e validam o resultado por meio dele.

A contribuição pretendida aqui é complementar. Em vez de construir mais um dispositivo, constrói-se o ambiente em que a cadeia de software pode ser validada antes e independentemente do dispositivo: com referência conhecida, falhas injetáveis, execuções repetíveis e um transporte intercambiável. A Tabela 1 resume a comparação.

Tabela 1: Comparação entre trabalhos centrados em hardware e esta plataforma

Aspecto	Trabalhos centrados em hardware	Esta plataforma
Foco	Dispositivo conectado ao veículo	Cadeia de software completa
Verdade de referência	Desconhecida em veículo real	Conhecida, gravada com cada leitura
Falhas de comunicação	Difíceis de provocar	Injetadas por comando, com semente
Repetibilidade	Depende do veículo e do momento	Determinística, por sementes
Custo para reproduzir	Hardware e veículo	Um computador, com ou sem Docker
Ponto de integração com hardware	Já é o produto	Interface Transport pronta

Fonte: elaborada pelos autores.

2.8 SÍNTESE

Os trabalhos revisados mostram que o OBD-II é um padrão estável e aberto, que o CAN é a base de transporte dominante, que a telemetria e a IoT oferecem modelos arquiteturais e protocolos de mensageria maduros, que a segurança exige cautela no acesso a interfaces de diagnóstico e que a simulação é prática estabelecida. A lacuna que sobra é um ambiente aberto, pequeno e repetível para validar a cadeia de software da telemetria OBD-II antes de ligá-la a um veículo. É essa lacuna que o restante do texto procura preencher.

3 FUNDAMENTAÇÃO TÉCNICA

Este capítulo apresenta os conceitos e as tecnologias necessários para entender a plataforma. Ele segue a ordem em que uma requisição percorre o sistema: do protocolo de diagnóstico (OBD-II) à rede (CAN), depois à camada de transporte (ISO-TP), ao adaptador (ELM327) e, por fim, às tecnologias de software que armazenam, distribuem e mostram os dados. A justificativa de cada escolha entre alternativas fica para o Capítulo 4.

3.1 DIAGNÓSTICO DE BORDO OBD-II

O OBD-II é definido por um conjunto de documentos que se complementam. A SAE J1962 especifica o conector de diagnóstico de 16 pinos e sua pinagem (SAE INTERNATIONAL, 2016a). A SAE J1979 define os modos de serviço de diagnóstico e os identificadores de parâmetros, com suas fórmulas de conversão (SAE INTERNATIONAL, 2017). A SAE J2012 define a estrutura e o significado dos códigos de falha (SAE INTERNATIONAL, 2016b). A ISO 15765-4 especifica como esses serviços trafegam sobre CAN em veículos com sistemas relacionados a emissões (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2021).

3.1.1 Modos de serviço

A comunicação é do tipo requisição e resposta. Uma ferramenta (o testador) envia uma requisição cujo primeiro byte é o **modo** de serviço. A ECU responde com o mesmo modo somado de `0x40`, de modo que a resposta ao modo `0x01` começa com `0x41`. Quando a requisição é inválida, a resposta negativa começa com `0x7F`, seguida do modo e de um código de motivo. A Tabela 2 resume os modos implementados neste trabalho.

Tabela 2: Modos de serviço OBD-II implementados

Modo	Requisição	Resposta	Finalidade
01	01 PID	41 PID dados	Dados atuais do motor
02	02 PID 00	42 PID 00 dados	Dados congelados no instante de uma falha
03	03	43 N e N códigos de 2 bytes	Códigos de falha armazenados
04	04	44	Limpa códigos e dados congelados
07	07	47 N e N códigos	Códigos de falha pendentes
09	09 02	49 02 01 e 17 bytes	Número de identificação do veículo (VIN)

Fonte: docs/spec/04-protocolo.md, a partir de SAE International (2017).

3.1.2 Identificadores de parâmetros (PIDs)

No modo 01, o segundo byte da requisição é o PID, que escolhe a grandeza. A resposta traz o PID repetido, seguido de 1 a 4 bytes de dados que o testador converte em valor físico por uma fórmula do padrão. A Tabela 3 lista os PIDs implementados, com a fórmula e a resolução, isto é, o menor passo que o byte transmitido consegue representar.

Tabela 3: PIDs implementados, fórmulas e resoluções

PID	Grandeza	Bytes	Fórmula	Unidade	Resolução
0x04	Carga calculada	1	A·100/255	%	0,392
0x05	Temperatura do arrefecimento	1	A − 40	°C	1
0x06	Ajuste de combustível de curto prazo	1	(A − 128)·100/128	%	0,781
0x0B	Pressão do coletor (MAP)	1	A	kPa	1
0x0C	Rotação do motor	2	(256A + B)/4	rpm	0,25
0x0D	Velocidade	1	A	km/h	1
0x0F	Temperatura do ar de admissão	1	A − 40	°C	1
0x11	Posição da borboleta	1	A·100/255	%	0,392
0x2F	Nível de combustível	1	A·100/255	%	0,392
0x42	Tensão do módulo de controle	2	(256A + B)/1000	V	0,001

Fonte: docs/spec/04-protocolo.md, a partir de SAE International (2017).

Os PIDs `0x00`, `0x20` e `0x40` têm função especial: devolvem uma máscara de 32 bits que informa quais PIDs de cada faixa de 32 a ECU suporta, com o bit mais significativo correspondendo ao primeiro PID da faixa. O último bit da máscara indica se há suporte a algum PID da faixa seguinte. É assim que uma ferramenta descobre o que pode perguntar, em vez de tentar às cegas. O PID `0x01` informa o estado do monitor de diagnóstico: o bit 7 do primeiro byte indica se a MIL está acesa e os bits 0 a 6 contam os códigos de falha armazenados.

Como a resposta carrega um número inteiro de 8 ou 16 bits, a conversão perde informação: um valor físico qualquer é arredondado para o passo mais próximo. O erro máximo de decodificação é, portanto, a metade da resolução, e esse limite teórico é o critério usado neste trabalho para dizer se uma leitura está correta (Seção 4.1, requisito RF06).

3.1.3 Códigos de falha (DTC)

Um código de falha ocupa 2 bytes. Os dois bits mais altos do primeiro byte escolhem o sistema (`00` para *Powertrain*, letra P; `01` para *Chassis*, C; `10` para *Body*, B; `11` para *Network*, U), os dois bits seguintes dão o primeiro dígito (0 a 3) e os 12 bits restantes formam três dígitos hexadecimais. Assim, os bytes `03 00` representam o código P0300, “falha de ignição aleatória ou em múltiplos cilindros” (SAE INTERNATIONAL, 2016b). Quando um código é armazenado, a ECU também captura os valores dos PIDs naquele instante, os chamados dados congelados (*freeze frame*), acessíveis pelo modo 02.

3.2 REDE CAN

O CAN transmite mensagens em quadros. No formato base, o quadro tem um identificador de 11 bits, um campo de controle com o comprimento dos dados (DLC, de 0 a 8), até 8 bytes de dados, um CRC e campos de confirmação e delimitação (BOSCH, 1991; INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2015). O identificador não nomeia um destinatário: ele nomeia o **conteúdo** da mensagem e define a prioridade. Quando dois nós tentam transmitir ao mesmo tempo, vence aquele cujo identificador tem o menor valor numérico, sem perda do quadro vencedor.

No OBD-II sobre CAN de 11 bits e 500 kbit/s, os identificadores têm papéis fixos (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2021):

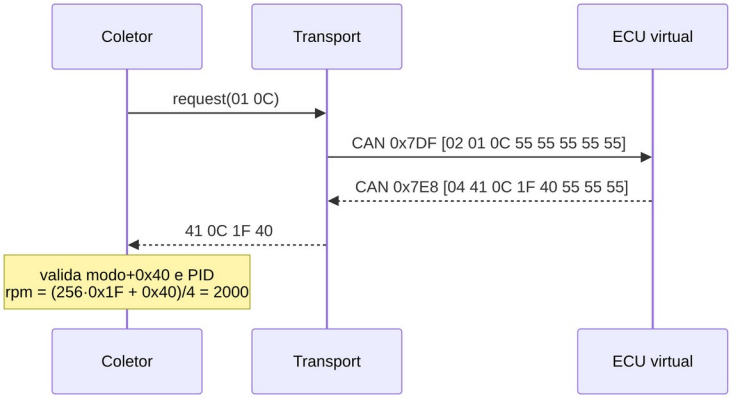
- `0x7DF`: requisição funcional, endereçada a todas as ECUs ao mesmo tempo;
- `0x7E0` a `0x7E7`: requisições físicas, endereçadas a uma ECU específica;
- `0x7E8` a `0x7EF`: respostas, em que `0x7E8` é, por convenção, a ECU do motor.

Os quadros do OBD-II têm sempre 8 bytes, com preenchimento de `0x55` nos bytes não usados. Em 500 kbit/s, cada bit dura 2 µs. O comprimento de um quadro de dados com 8 bytes no pior caso (com inserção de bits de preenchimento) é de 135 bits, ou 270 µs, valor obtido pela expressão da literatura para o comprimento máximo de um quadro, $C = 47 + 8s + \lfloor (34 + 8s - 1)/4 \rfloor$ bits, com $s = 8$ (DAVIS et al., 2007). Esse é o tempo que o barramento físico gastaria por quadro; o barramento virtual deste trabalho não o reproduz.

A norma ainda impõe um prazo ao testador. O tempo máximo de resposta da ECU à requisição, chamado P2, é de 50 ms. Respostas além desse prazo são tratadas como ausência de resposta.

A Figura 1 mostra uma consulta completa de rotação. O testador envia o quadro `02 01 0C` (2 bytes úteis, modo 01, PID 0C) ao identificador `0x7DF`. A ECU responde em `0x7E8` com `04 41 0C 1F 40` (4 bytes úteis, modo 41, PID 0C, dados `1F 40`). A rotação é (256 · 31 + 64)/4 = 2000 rpm.

Figura 1: Consulta de rotação (modo 01, PID 0C) sobre CAN



Fonte: elaborada pelos autores (diagrama gerado de docs/tcc/img/fig-seq.mmd).

3.3 TRANSPORTE ISO-TP

Um quadro CAN carrega no máximo 8 bytes, e algumas respostas OBD são maiores. O VIN, por exemplo, tem 17 caracteres. A camada de transporte ISO-TP, definida na ISO 15765-2 (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2016b), resolve isso segmentando uma mensagem em vários quadros e remontando-a no destino. O primeiro byte do campo de dados de cada quadro é a informação de controle de protocolo (PCI), e seu primeiro nibble define o tipo do quadro, conforme a Tabela 4.

Tabela 4: Tipos de quadro do ISO-TP

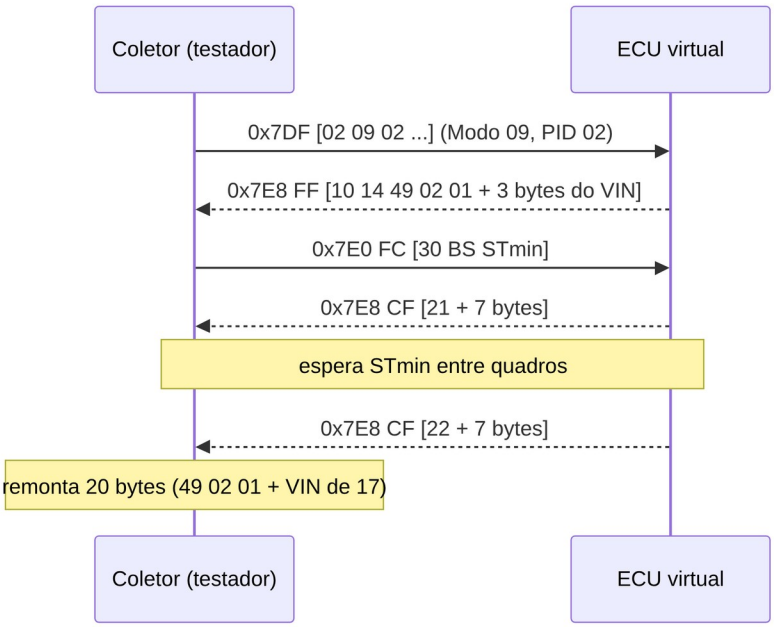
Tipo	Byte PCI	Conteúdo
Quadro único (SF)	0x0N	N bytes úteis (1 a 7)
Primeiro quadro (FF)	0x1L LL	Comprimento total em 12 bits e 6 bytes de dados
Quadro consecutivo (CF)	0x2S	Número de sequência S (0 a F) e 7 bytes de dados
Controle de fluxo (FC)	0x30 BS STmin	Autoriza o envio: tamanho do bloco e espaçamento mínimo

Fonte: docs/spec/04-protocolo.md, a partir da ISO 15765-2.

Mensagens de até 7 bytes cabem em um quadro único. Mensagens maiores começam com um primeiro quadro, que informa o comprimento total. Quem recebe responde com um quadro de controle de fluxo, em que **BS** (*block size*) diz quantos quadros consecutivos podem vir antes do próximo controle de fluxo (0 significa “sem limite”) e **STmin** diz o tempo mínimo entre quadros consecutivos. A remontagem tem um prazo: se um quadro consecutivo não chegar em 1 s (parâmetro N_Cr), a recepção é abortada.

A Figura 2 mostra a leitura do VIN (modo 09, PID 02). A resposta tem 20 bytes: `49 02 01` seguidos dos 17 caracteres. O primeiro quadro carrega 6 bytes (os 3 do cabeçalho e 3 caracteres do VIN) e dois quadros consecutivos carregam 7 bytes cada, totalizando 6 + 7 + 7 = 20 bytes.

Figura 2: Leitura do VIN por ISO-TP (primeiro quadro, controle de fluxo e dois quadros consecutivos)



Fonte: elaborada pelos autores (diagrama gerado de docs/tcc/img/fig-isotp.mmd).

3.4 ADAPTADOR ELM327

O ELM327 é um circuito integrado da Elm Electronics que faz a ponte entre uma interface serial e os protocolos OBD-II (ELM ELECTRONICS, [s.d.]). O computador ou o aplicativo não monta quadros CAN: envia linhas de texto, e o chip faz o resto. Há duas famílias de comandos. Os comandos **AT** configuram o adaptador (`ATZ` reinicia, `ATE0` desliga o eco, `ATSP0` escolhe o protocolo automático, `ATH1` liga a exibição de cabeçalhos, `ATDPN` informa o protocolo em uso). Qualquer linha que

não comece com AT é enviada ao veículo como requisição OBD em hexadecimal, por exemplo `010C`. O chip responde com os bytes da resposta em hexadecimal e termina com o caractere de prompt `>`.

Adaptadores Wi-Fi baseados nesse chip costumam criar uma rede própria e aceitar conexões TCP em um endereço e porta típicos (192.168.0.10 e 35000). Esse arranjo é importante porque transforma o adaptador em um servidor de texto simples sobre TCP, que qualquer cliente em qualquer sistema consegue usar. Bibliotecas como a python-OBD (PYTHON-OBD, [s.d.]) e aplicativos de celular e de computador se apoiam nesse dialeto. Ao fazer o simulador falar o mesmo dialeto, ganha-se acesso a todo esse ecossistema como clientes de teste.

Na prática, o dialeto tem sutilezas que não estão no padrão OBD-II: o formato das linhas com cabeçalho ligado (`7E8 06 41 00 9C 3A 80 01`, com identificador, byte de comprimento do ISO-TP e dados), as respostas `NO DATA` e `SEARCHING...`, a lista de comandos AT que um cliente espera ver aceitos. Essas sutilezas foram descobertas, no desenvolvimento deste trabalho, justamente ao testar o simulador contra clientes de terceiros (Seção 5.7).

3.5 TECNOLOGIAS DE SOFTWARE

3.5.1 Python e programação assíncrona

Python 3.12 ou superior é a linguagem de toda a plataforma. A biblioteca padrão `asyncio` permite executar muitas tarefas que passam a maior parte do tempo esperando entrada e saída (rede, temporizadores) em um único fio de execução, com a sintaxe `async` e `await`. O coletor, o simulador e o servidor são, em boa parte, esse tipo de tarefa.

3.5.2 python-can

A biblioteca `python-can` oferece uma abstração de barramento CAN com várias interfaces de *hardware* e uma interface `virtual`, que simula um barramento em memória entre objetos do mesmo processo (PYTHON-CAN, [s.d.]). Trocar a interface virtual por `socketcan` ou `slcan` muda a configuração, não o código.

3.5.3 FastAPI, Uvicorn e Pydantic

O FastAPI é um arcabouço para construir APIs em Python com tipagem e validação de dados a partir de anotações de tipo (FASTAPI, [s.d.]). Ele gera documentação OpenAPI automaticamente e suporta WebSocket. O Uvicorn é o servidor ASGI que o executa, e o Pydantic valida os esquemas JSON das mensagens.

3.5.4 HTTP, WebSocket e MQTT

O HTTP é o protocolo de requisição e resposta da web (FIELDING; NOTTINGHAM; RESCHKE, 2022). O WebSocket abre um canal bidirecional e persistente sobre uma conexão HTTP, adequado a fluxos em tempo real (FETTE; MELNIKOV, 2011). O MQTT é um protocolo de publicação e assinatura com um intermediário, em que os assuntos são tópicos hierárquicos como `telemetria/<veiculo>/dados`, e em que cada publicação escolhe um nível de QoS (OASIS, 2014). O QoS 0 entrega no máximo uma vez, sem confirmação; o QoS 1 entrega ao menos uma vez, com confirmação.

3.5.5 SQLite e SQLAlchemy

O SQLite é um banco de dados relacional embutido, que guarda tudo em um único arquivo e dispensa um processo servidor (SQLITE, [s.d.]). O SQLAlchemy 2 é o mapeador objeto-relacional, usado aqui em modo assíncrono.

3.5.6 Front-end com HTML, JavaScript e SVG

O dashboard e o painel de controle são páginas HTML com JavaScript puro, sem estrutura de construção e sem bibliotecas externas. Os mostradores (conta-giros e manômetro) são desenhados em SVG, formato vetorial que o navegador renderiza e que pode ser animado por script.

3.5.7 Docker e Docker Compose

O Docker empacota uma aplicação e suas dependências em uma imagem, executada como contêiner isolado do sistema anfitrião. O Docker Compose descreve, em um arquivo, vários contêineres e como se conectam (DOCKER, [s.d.]).

Aqui ele serve a dois propósitos: executar a plataforma com um comando e isolar as ferramentas de desenvolvimento, de modo que nada precise ser instalado no computador do autor além do próprio Docker.

3.5.8 Testes: pytest e hypothesis

O `pytest` é o executor de testes usado em todos os níveis. O `hypothesis` implementa testes baseados em propriedades: em vez de exemplos fixos, o desenvolvedor descreve uma propriedade que deve valer para qualquer entrada (por exemplo, “decodificar o que foi codificado devolve o valor original dentro da resolução”) e a biblioteca gera muitos casos, inclusive os de borda.

3.6 SÍNTESE

O percurso de uma leitura é o seguinte: um aplicativo ou o coletor escolhe um modo e um PID; a requisição vira um quadro CAN de 8 bytes em `0x7DF`; a ECU responde em `0x7E8`, usando ISO-TP quando a resposta é maior que 7 bytes; o testador converte os bytes em valor físico pela fórmula do PID; o valor é publicado, armazenado e exibido. Um adaptador ELM327 esconde os quadros e expõe esse mesmo percurso como texto por TCP. As próximas seções mostram como cada etapa foi construída.

4 MATERIAIS E MÉTODOS

Este capítulo descreve o que foi construído e como. Ele começa pelos requisitos e pela arquitetura, justifica as escolhas de tecnologia frente às alternativas, detalha cada módulo (protocolo, simulador, transporte, coletor, servidor e interfaces) e termina com a estratégia de testes, o ambiente de execução, o processo de desenvolvimento e o desenho dos ensaios.

4.1 REQUISITOS

Os requisitos foram escritos antes do código e cada um tem um critério de aceitação verificável. A Tabela 5 reúne os requisitos funcionais e a Tabela 6 os não funcionais.

Tabela 5: Requisitos funcionais (RF) e critérios de aceitação

ID	Requisito	Critério de aceitação
RF01	O simulador responde ao modo 01 para os PIDs configurados	Resposta em 0x7E8 com modo + 0x40 e PID repetido
RF02	Descoberta de PIDs suportados (0x00, 0x20, 0x40)	Máscara de 32 bits correta; o coletor pede só PIDs suportados
RF03	Leitura de DTCs (modo 03), pendentes (modo 07) e limpeza (modo 04)	P0300 injetado aparece em até 2 ciclos de consulta; modo 04 zera a lista
RF04	Dados congelados (modo 02) ao surgir uma falha	Freeze frame com os PIDs do instante da falha
RF05	ISO-TP completo (primeiro quadro, consecutivos, controle de fluxo)	VIN de 17 caracteres lido pelo modo 09, PID 0x02
RF06	O coletor requisita, valida e decodifica PIDs pelas fórmulas da SAE J1979	Erro máximo ≤ metade da resolução em 100% das leituras
RF07	O servidor recebe a telemetria e a persiste	Nenhuma perda ou duplicação, verificada pelo número de sequência
RF08	API HTTP de consulta e WebSocket de tempo real	O dashboard atualiza sem recarregar
RF09	O painel de controle altera o simulador em tempo de execução	Mudança refletida na leitura em até 2 ciclos
RF10	Camada ELM327 por TCP	Um cliente ELM327 comum completa ATZ, ATE0, 0100 e 010C
RF11	Injeção de falhas de comunicação (atraso, perda, resposta inválida, sem resposta)	O coletor se recupera sem travar e registra o evento
RF12	Transporte plugável por configuração	Trocar o simulador por ELM327/TCP sem alterar o código do coletor

Fonte: docs/spec/01-visao-requisitos.md.

Tabela 6: Requisitos não funcionais (RNF) e metas

ID	Requisito	Meta
RNF01	Tempo de ciclo de 10 PIDs	≤ 150 ms com a ECU em 10 ms
RNF02	Latência do coletor ao banco	p95 ≤ 50 ms
RNF03	Perda de mensagens	0 em ensaio de 240 s

ID	Requisito	Meta
RNF04	Conformidade com a ISO 15765-4	11 bits, 500 kbit/s, P2 = 50 ms
RNF05	Reprodutibilidade	Sementes fixas e ensaios repetíveis
RNF06	Cobertura de testes	≥ 80% nos módulos de protocolo
RNF07	Instalação	Um comando (docker compose up ou make run)
RNF08	Segurança	Somente leitura; API local, sem exposição externa por padrão

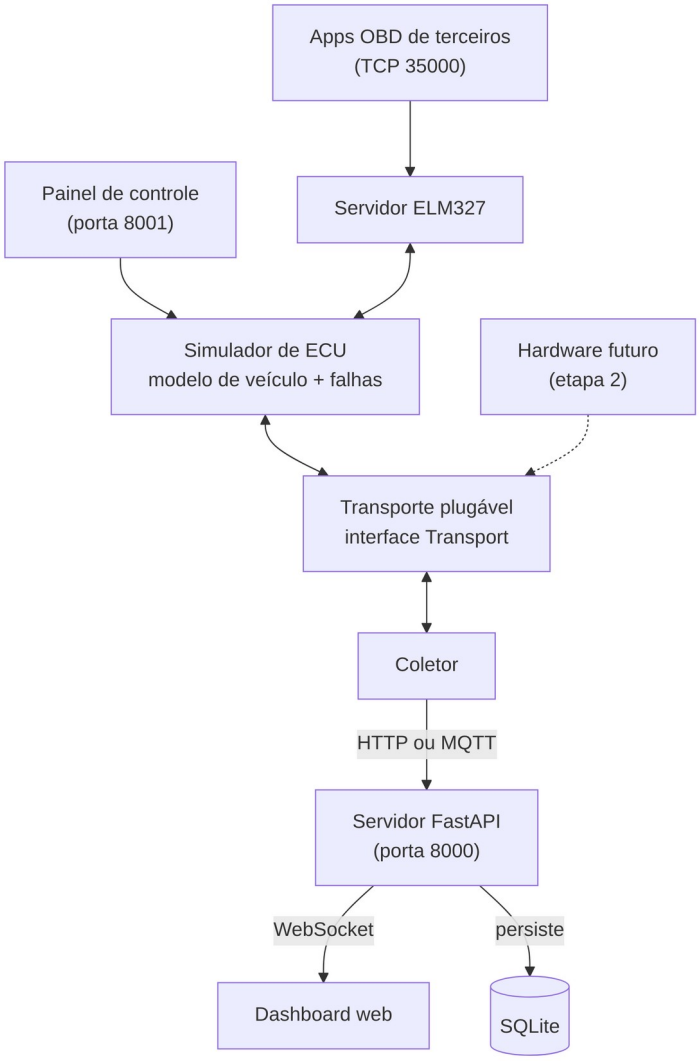
Fonte: docs/spec/01-visao-requisitos.md.

O RNF05 originalmente pedia repetição em duas máquinas. Como este trabalho mediu apenas uma máquina, a parte de reprodutibilidade que foi cumprida é a determinística (mesmas sementes, mesmos valores físicos e mesmos erros de decodificação), e a comparação de tempos entre máquinas é declarada como pendente (Seção 5.9).

4.2 ARQUITETURA

A arquitetura obedece a uma regra simples: o coletor só conhece a interface `Transport`. O que existe do outro lado dessa interface (um simulador, um barramento CAN virtual, um adaptador ELM327 por TCP ou, na segunda etapa, um microcontrolador) é detalhe de implementação. A Figura 3 mostra os blocos e as conexões.

Figura 3: Arquitetura da plataforma



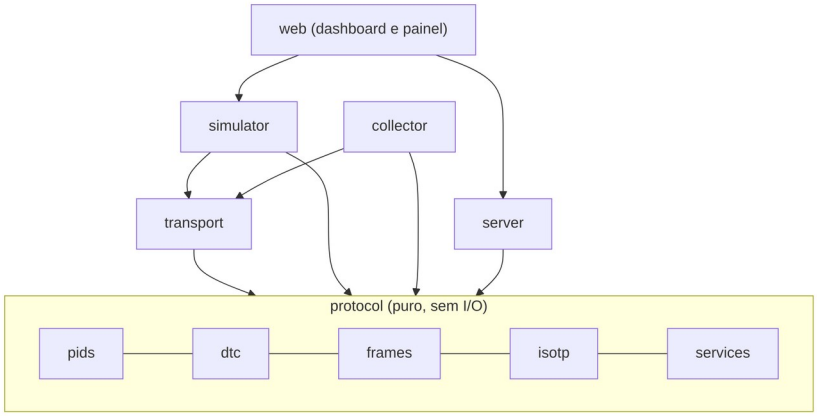
Fonte: elaborada pelos autores (docs/tcc/img/fig-arq.mmd).

O fluxo de dados em operação normal tem cinco passos. O painel de controle altera o simulador por HTTP. O coletor pede PIDs ao transporte e a ECU virtual responde. O coletor publica uma amostra (`Sample`) por HTTP ou MQTT. O servidor grava a amostra no SQLite e a repassa por WebSocket. O dashboard desenha os valores e o estado dos códigos de falha.

4.2.1 Módulos e dependências

O código está organizado em seis módulos sob `src/obd/`, com dependências em um único sentido (Figura 4). O núcleo `protocol` não faz entrada e saída e não depende de nenhum outro módulo. Essa restrição é deliberada: ela permite testar o protocolo sem rede, sem relógio e sem processos, e garante que o que foi validado como “protocolo” não está contaminado por detalhes de infraestrutura.

Figura 4: Módulos e sentido das dependências



Fonte: elaborada pelos autores (docs/tcc/img/fig-camadas.mmd).

A Tabela 7 resume a responsabilidade de cada módulo e o tamanho aproximado do código-fonte em linhas.

Tabela 7: Módulos, responsabilidades e tamanho do código-fonte

Módulo	Responsabilidade	Depende de	Linhas
protocol	Fórmulas, quadros, ISO-TP, serviços, DTCs	nada	668
transport	Enviar e receber quadros OBD	protocol	333
simulator	ECU virtual, modelo de veículo, falhas, servidor ELM327, API de controle	protocol, transport	1216
collector	Requisitar, validar, decodificar e publicar	protocol, transport	494
server	API, WebSocket, persistência	protocol (esquemas)	793
web	Dashboard e painel de controle (HTML, CSS e JavaScript)	API do server e do simulator	841

Fonte: contagem de linhas dos arquivos `.py`, `.js`, `.html` e `.css` de cada pasta de `src/obd/`, em 24/10/2026. Não inclui `runner.py`, `mqtt_common.py` e testes.

4.2.2 Portas e processos

Em operação, a plataforma expõe quatro endpoints (Tabela 8). O comando `python -m obd run` sobe servidor, simulador, painel, servidor ELM327 e coletor em um único processo, como tarefas assíncronas; a mesma instância de transporte é compartilhada entre o servidor ELM327 e o coletor, de modo que a semente e as falhas valem para ambos.

Tabela 8: Portas e serviços

Serviço	Porta	Protocolo	Quem se conecta
Servidor da plataforma	8000	HTTP e WebSocket	Navegador e coletor
Simulador (controle)	8001	HTTP e WebSocket	Painel de controle
Simulador (ELM327)	35000	TCP	Aplicativos e bibliotecas OBD de terceiros
Mosquitto (opcional)	1883	MQTT	Coletor e servidor

Fonte: docs/spec/05-api.md.

4.3 ESCOLHAS DE TECNOLOGIA

Cada escolha foi feita a partir de um critério explícito: reduzir o atrito para quem quiser executar, entender e estender a plataforma, sem sacrificar o que os requisitos exigem. A Tabela 9 mostra a escolha, as alternativas que seriam plausíveis e o motivo. As alternativas são discutidas por raciocínio de projeto; elas não foram implementadas nem comparadas experimentalmente, com a exceção do canal HTTP contra MQTT, que é medido no ensaio E7.

Tabela 9: Escolhas de tecnologia, alternativas consideradas e motivos

Camada	Escolha	Alternativas plausíveis	Motivo
Linguagem	Python 3.12+	C/C++, Go, Rust, JavaScript	Ecossistema para CAN (python-can), web e testes; tipagem completa com mypy; permite reaproveitar o ambiente de trabalho já usado pelos autores
Concorrência	asyncio	Threads, processos, bibliotecas externas	Muitas tarefas que esperam entrada e saída; o coletor, o simulador e o servidor rodam no mesmo processo ou separados
Servidor web	FastAPI e Uvicorn	Flask, Django	HTTP, WebSocket e OpenAPI automáticos; validação por tipos
CAN virtual	python-can, interface virtual	SocketCAN vcan (só Linux), barramento próprio	Barramento em memória multiplataforma; trocável por socketcan ou slcan apenas por configuração
Banco	SQLite via SQLAlchemy 2	PostgreSQL, bancos de séries temporais	Zero configuração, arquivo único, simples de reproduzir; volume de dados pequeno
Mensageria	HTTP por padrão; MQTT com Mosquitto opcional	Somente MQTT, Kafka, CoAP	HTTP não exige infraestrutura extra; MQTT é o padrão de fato em IoT e é medido contra o HTTP
Tempo real no navegador	WebSocket	Consultas periódicas, *Server-Sent Events*	Canal bidirecional e persistente; o servidor empurra amostras sem consulta
Front-end	HTML e JavaScript puro, SVG	React, Vue, bibliotecas de gráficos	Funciona sem etapa de construção e sem rede; poucas dependências
Testes	pytest, pytest-asyncio, hypothesis	unittest, nose	Testes simples, assíncronos e baseados em propriedades
Qualidade	ruff, mypy (modo estrito), pre-commit	flake8, pylint	Verificação rápida e tipagem completa como porta de entrada
Execução	Docker Compose e Makefile	Instalação manual, ambientes virtuais	Um comando (RNF07) e isolamento das ferramentas

Fonte: docs/spec/02-arquitetura-stack.md, ampliado pelos autores.

4.3.1 Por que simulador, e não apenas uma gravação de dados reais

Uma alternativa a um simulador é reproduzir dados gravados de um veículo. Ela foi descartada por três razões. Primeiro, dados gravados trazem o que o veículo

disse, mas não o valor verdadeiro; o que se mede é a fidelidade da cópia, não a exatidão da decodificação. Segundo, é impossível provocar, em uma gravação, uma falha que não aconteceu. Terceiro, gravações não respondem a requisições: o coletor precisa de uma ECU que reaja ao que ele pergunta, inclusive ao que não deveria perguntar.

4.3.2 Por que um transporte plugável

O requisito RF12 existe para transformar a validação em software em um ponto de partida para o hardware. Se o coletor dependesse de um simulador específico, a segunda etapa exigiria reescrevê-lo. Com a interface `Transport`, uma nova classe é suficiente. O contrato é pequeno (Seção 4.6) e foi mantido estável durante todo o desenvolvimento.

4.3.3 Por que o ELM327 como camada de compatibilidade

Há duas formas de aproximar o simulador do mundo real: reproduzir o CAN ou reproduzir o adaptador. A primeira é o que um microcontrolador com transceptor faria, e pertence à segunda etapa. A segunda é possível em software puro e traz um ganho imediato: qualquer biblioteca ou aplicativo OBD que fale com adaptadores ELM327 passa a ser um cliente de teste independente, escrito por terceiros, sem conhecimento do nosso código. Isso reduz o risco de um erro de interpretação do padrão que passaria despercebido em testes que escrevemos nós mesmos.

4.4 NÚCLEO DE PROTOCOLO

O módulo `protocol` contém cinco arquivos e 668 linhas de código-fonte. Todos são funções e estruturas puras: sem entrada e saída, sem `time.time()` e sem aleatoriedade global.

- `pids.py` descreve cada PID com uma estrutura imutável (`PidDef`): código, nome, unidade, número de bytes, resolução e funções `decode` e `encode`. A função `encode` é usada pela ECU simulada, e `decode` pelo coletor; assim, um teste pode verificar que `decode(encode(x))` volta a x dentro de metade da resolução.
- `dtc.py` converte entre os 2 bytes do DTC e o texto (`03 00` e `P0300`), e traz as descrições em português de P0300 a P0304, P0171, P0172, P0420 e P0133.

- `frames.py` monta e valida quadros CAN de 11 bits: identificador (`0x7DF` e `0x7E0` a `0x7E8`) e DLC igual a 8.
- `isotp.py` implementa segmentação e remontagem (SF, FF, CF, FC), com um remontador e um emissor que recebem o tempo por parâmetro, de modo que prazos (N_Cr) e espaçamentos (STmin) são testáveis sem esperar.
- `services.py` constrói requisições e interpreta respostas dos modos 01, 02, 03, 04, 07 e 09, validando o eco de modo e de PID e reconhecendo respostas negativas (`7F`).

Algumas decisões de detalhe merecem registro, porque aparecem nos resultados. A resolução de cada PID usa o passo exato (100/255 e 100/128) e não o valor arredondado da norma (0,392 e 0,781); com o valor arredondado, o erro de ida e volta poderia ultrapassar metade da resolução por um problema de arredondamento do critério, e não do código. O STmin com valores reservados (`0x80` a `0xF0` e `0xFA` a `0xFF`) é tratado como 127 ms, conforme a recomendação da norma. O prazo N_Cr de 1 s reinicia a cada quadro consecutivo e vale também para o emissor esperando o controle de fluxo.

4.5 SIMULADOR

O simulador tem três partes: o modelo de veículo, a ECU virtual e o injetor de falhas.

4.5.1 Modelo de veículo

O modelo avança em passos fixos de 10 ms e mantém o **estado físico real**, que é a referência para medir o erro de decodificação. Os valores representam, de forma ilustrativa, um compacto 1.6 aspirado, e podem ser trocados em `config/default.yaml`. A Tabela 10 mostra como cada variável é calculada.

Tabela 10: Variáveis do modelo de veículo

Variável	Como é calculada
Velocidade	Segue o perfil de condução com aceleração máxima de 10 km/h/s e desaceleração máxima de 15 km/h/s
Marcha	Trocas em 25, 45, 65 e 85 km/h
Rotação	Pela velocidade, marcha, relação final e raio do pneu; marcha lenta em 800 rpm e corte em 6500 rpm
Borboleta	Pela aceleração pedida pelo perfil
Carga e MAP	Carga = 15 + 85·borboleta·(0,4 + 0,6·rpm/6000); MAP = min(101, 28 + 0,73·carga), limitado à pressão atmosférica por ser motor aspirado

Variável	Como é calculada
Temp. do arrefecimento	Aproximação exponencial de 25 °C até 90 °C, com constante de tempo de 120 s
Temp. do ar de admissão	Ambiente mais um acréscimo lento com a temperatura do motor
Ajuste de mistura (STFT)	Oscila em torno de 0% com amplitude de 3%, simulando a malha fechada
Nível de combustível	Decresce proporcionalmente à carga integrada
Tensão	14,1 V com o motor ligado
Ruído	Gaussiano de baixa amplitude, com semente fixa

Fonte: docs/spec/06-simulador.md e docs/duvidas.md (tarefa 04).

A rotação em marcha engatada segue a relação entre velocidade e rotação das rodas:

$$n_{rpm} = \frac{v/3{,}6}{r_{pneu}} \cdot i_{marcha} \cdot i_{final} \cdot \frac{60}{2\pi}$$

em que v é a velocidade em km/h, r_pneu é o raio do pneu em metros, i_marcha e i_final são as relações de transmissão. Os valores padrão são raio de 0,30 m, relação final de 4,06 e relações de marcha de 3,62, 2,19, 1,41, 1,03 e 0,81.

Os **perfis de condução** são arquivos YAML com degraus de velocidade-alvo e borboleta no tempo. Cinco são entregues: `parado`, `urbano` (marcha lenta, alvo de 50 km/h a partir de 10 s, cruzeiro e parada aos 50 s), `rodovia`, `ciclo_120s` (marcha lenta, 0 a 80 km/h, cruzeiro e desaceleração) e `esforco` (rotação alta, para exercitar os PIDs rápidos). O ruído gaussiano entra no valor reportado (o que a ECU codifica), e não no estado físico limpo.

4.5.2 ECU virtual

A ECU é uma máquina de estados: recebe quadros (`on_frame`) e devolve os quadros devidos (`poll`), com o relógio injetado. Isso torna o atraso de resposta testável sem esperar de fato (um `FakeClock` avança o tempo). Ela responde a todos os modos da Tabela 2, anuncia na descoberta apenas os PIDs que realmente suporta (e o bit de continuação só se houver PID acima da faixa), captura o *freeze frame* ao armazenar o primeiro DTC e responde às requisições inválidas com `7F`: modo desconhecido (`7F modo 11`), PID do modo 09 diferente de 02 (`7F 09 12`) e tamanho incorreto (`7F modo 13`). Um PID do modo 01 não suportado recebe silêncio, que é o comportamento esperado na ISO 15765-4.

Para cada resposta com valor físico, a ECU registra o par (número da requisição, valor verdadeiro). O coletor nunca tem acesso a esse registro: ele existe

apenas para o executor de ensaios, que o junta à amostra publicada pelo número de sequência e o grava em `readings.true_value`.

4.5.3 Injeção de falhas

O injetor (`faults.py`) implementa as falhas da Tabela 11. Todas as decisões aleatórias vêm de um gerador com semente própria, e uma probabilidade igual a zero não consome a semente, o que preserva a sequência quando a falha está desligada.

Tabela 11: Catálogo de falhas do simulador

Falha	Parâmetros	O que exercita
DTC armazenado	código, instante	Modo 03, MIL, freeze frame
DTC pendente	código	Modo 07
Atraso de resposta	mínimo e máximo em ms	Sensibilidade do ciclo (ensaio E2)
Perda de quadros	probabilidade	Timeout e recuperação (ensaio E4)
Quadro corrompido	probabilidade	Validação do eco de modo e de PID
ECU muda	liga e desliga	Recuo progressivo
PID não suportado	lista	Descoberta de PIDs
Resposta negativa	modo e código	Tratamento de 7F
Atraso de ISO-TP	STmin e BS	Remontagem multiquadro (ensaio E5)

Fonte: docs/spec/06-simulador.md.

O determinismo é um princípio transversal. O simulador recebe duas sementes, uma para o modelo e outra para a ECU e as falhas, e o relógio é injetável. Com as mesmas sementes e o mesmo perfil, a sequência de valores físicos é idêntica entre execuções. O modelo avança em passos fixos de 10 ms até o instante atual do relógio, de modo que o resultado não depende de quantas vezes ou com que granularidade ele é consultado.

4.6 TRANSPORTE E CAMADA ELM327

4.6.1 Interface `Transport`

O contrato entre o coletor e o que está do outro lado é uma interface pequena:

```
class Transport(Protocol):
    async def open(self) -> None: ...
    async def close(self) -> None: ...
    async def request(self, payload: bytes, timeout_ms: int = 100) -> bytes |
None:
        """Envia uma requisição OBD (ex.: 01 0C) e devolve a resposta completa
        já remontada pelo ISO-TP. None se estourar o tempo."""
```

```
@property
def name(self) -> str: ...
```

O método `request` recebe a mensagem de aplicação (por exemplo, `01 0C`) e devolve a resposta de aplicação já remontada (`41 0C 1F 40`). Tudo o que fica entre os dois (quadros, ISO-TP, controle de fluxo, prazos) é responsabilidade do transporte. O parâmetro `timeout_ms` vale até o primeiro quadro (o prazo P2 do testador); depois de um primeiro quadro, vale o N_Cr de 1 s.

4.6.2 Implementações

Três implementações estão disponíveis (Tabela 12). O `LoopbackTransport` liga o coletor diretamente à ECU em memória, e é o que os ensaios usam. O `CanVirtualTransport` envia quadros reais pelo barramento virtual do `python-can`. O `Elm327TcpTransport` abre um soquete TCP e fala o dialeto ELM327 com um servidor, que pode ser o do simulador ou, na prática, um adaptador real.

Tabela 12: Implementações da interface Transport

Transporte	Como os bytes chegam à ECU	Uso
loopback	Chamadas diretas em memória	Testes e ensaios; padrão do make run
can_virtual	Quadros no barramento virtual do python-can	Teste de integração com ISO-TP real
elm327_tcp	Texto por TCP (porta 35000)	Adaptadores e servidor ELM327 do simulador

Fonte: docs/spec/02-arquitetura-stack.md e docs/duvidas.md (tarefas 03 e 09).

4.6.3 Servidor ELM327

O `Elm327Server` fala o dialeto de texto sobre TCP e traduz cada comando OBD em uma chamada ao `Transport` do simulador. Assim, ISO-TP, atraso e falhas injetadas valem também para clientes ELM327. O servidor aceita os comandos AT de uso comum (`ATZ`, `ATE0/1`, `ATL`, `ATS`, `ATH0/1`, `ATSP`, `ATTP`, `ATDP`, `ATDPN`, `ATRV`, `ATAT`, `ATST`, `ATSH`, `ATCAF`, `ATAL`, `ATCRA` e outros), responde `ELM327 v1.5` ao `ATZ` e `ATI`, informa `ISO 15765-4 (CAN 11/500)` ao `ATDP` e `A6` ao `ATDPN`, e imprime, com `ATH1`, uma linha por quadro no formato `7E8 <PCI> <dados>`. Um serviço ou PID sem suporte, e também uma resposta negativa da ECU, viram `NO DATA`. Toda resposta termina com `\r\r>`, e a primeira consulta após `ATZ` pode vir precedida de `SEARCHING...`.

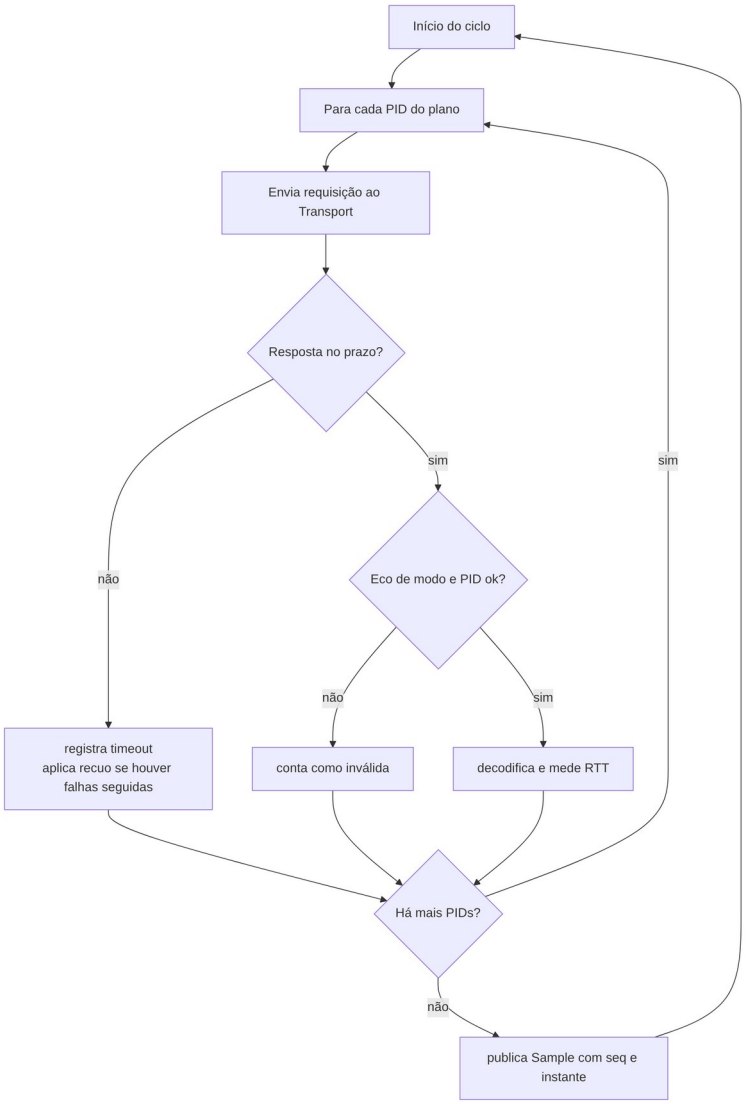
Esse formato não saiu pronto da especificação. A primeira versão do servidor foi escrita a partir do conhecimento do dialeto e passava nos testes escritos pelos próprios autores, mas falhou ao ser exercitada por um cliente de terceiros (Seção

5.7). As correções, e a variável de ambiente `OBD_ELM327_LOG=1`, que imprime cada comando recebido e a resposta enviada, foram resultado desse ciclo.

4.7 COLETOR

O coletor (`collector.py`, 494 linhas contando o agendador, o publicador e a fábrica) executa um laço de aquisição cujo fluxograma está na Figura 5.

Figura 5: Ciclo de aquisição do coletor



Fonte: elaborada pelos autores (docs/tcc/img/fig-ciclo.mmd).

Os comportamentos principais são:

1. **Descoberta.** Na abertura, o coletor consulta os PIDs `0x00`, `0x20` e `0x40`, segue o bit de continuação, e guarda a lista de PIDs suportados. Se nada for anunciado, tenta de novo no ciclo seguinte.

2. **Plano de leitura.** Os PIDs rápidos (rotação, MAP, borboleta e velocidade) são lidos a cada ciclo; os lentos (carga, temperaturas, combustível, tensão e ajuste de mistura) a cada 5 ciclos. O plano é configurável por PID, com `a_cada` e `prioridade`.
3. **Validação e decodificação.** Para cada PID, o coletor valida o eco de modo e de PID, decodifica pela fórmula e mede o RTT (tempo de ida e volta). Resposta com eco errado, tamanho errado ou negativa é contada como inválida; ausência de resposta, como *timeout*.
4. **Recuo progressivo.** Com n falhas seguidas acima de um limiar (3), antes de cada requisição o coletor espera mín(2000; 50 · 2^(n−3)) ms; qualquer resposta válida zera a contagem.
5. **Diagnóstico.** A cada 2 s, lê o PID `0x01` (MIL e contagem) e, se houver mudança, lê o modo 03. A primeira leitura sempre lê o modo 03.
6. **Publicação.** Ao fim do ciclo, publica uma amostra com número de sequência contínuo e instante. Um ciclo sem nenhuma leitura válida ainda publica uma amostra (vazia, com erros contados), para o número de sequência não ter lacunas.

Os relógios são injetáveis: o monotônico serve a RTT e ciclo, e o relógio de parede serve ao instante da amostra. O padrão usa `time.monotonic` e `time.time` apenas no coletor, nunca no protocolo.

4.7.1 Publicadores

A interface `Publisher` tem duas operações, `publish_sample` e `publish_dtcs`. O `HttpPublisher` envia por `POST` e o `MqttPublisher` publica nos tópicos `telemetria/<veiculo>/dados` (QoS 0, por privilegiar vazão em amostras repetitivas) e `telemetria/<veiculo>/dtc` (QoS 1, por serem eventos que não devem se perder). Se o *broker* estiver fora do ar, o `MqttPublisher` repete a mensagem pendente com recuo de 1, 2, 4 até 30 s, de modo que o número de sequência nunca pula nem inverte.

4.8 SERVIDOR, BANCO DE DADOS E INTERFACES

4.8.1 API do servidor

O servidor FastAPI escuta na porta 8000 e oferece as rotas da Tabela 13. A latência de cada amostra é a diferença entre o instante de recebimento (relógio do

servidor) e o instante de publicação (`ts`, relógio do coletor); a taxa de atualização é o número de amostras recebidas nos últimos 3 s dividido por 3. As perdas são as lacunas do número de sequência entre o menor e o maior da execução, e uma duplicata é descartada, responde `202` ao coletor e vira um evento `duplicate`.

Tabela 13: Rotas do servidor da plataforma

Método e rota	Função
POST /api/ingest/sample	Recebe uma amostra do coletor (resposta 202)
POST /api/ingest/dtc	Recebe um relatório de DTCs (resposta 202)
GET /api/latest	Última amostra
GET /api/history?since=&limit=	Histórico de amostras
GET /api/dtc	DTCs ativos e histórico
GET /api/stats	Contagens, latência, perdas e taxa de atualização
GET /api/export.csv?since=	Exportação para análise
GET /healthz	Verificação de saúde
WS /ws/stream	Fluxo em tempo real de amostras e DTCs

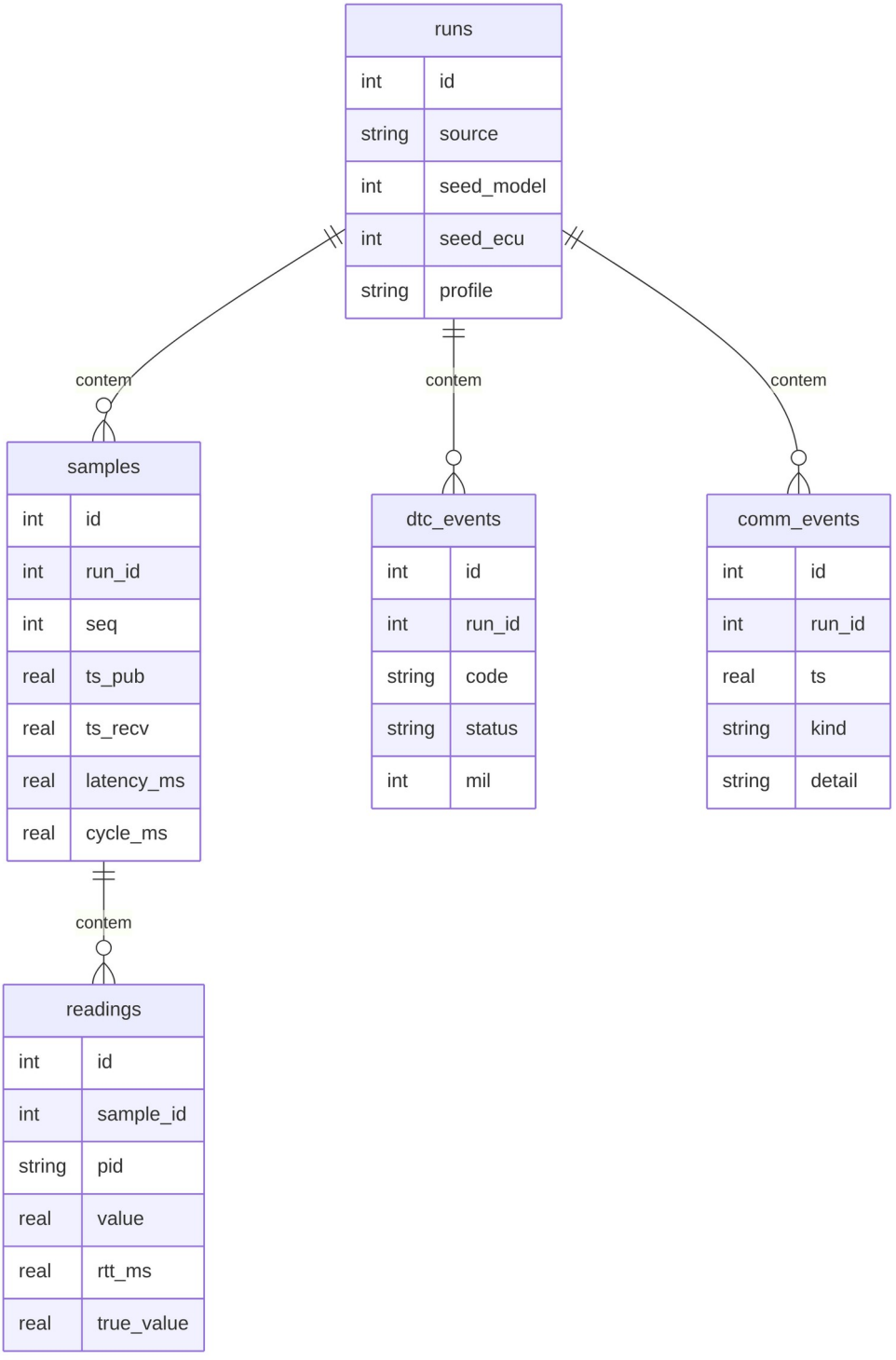
Fonte: docs/spec/05-api.md e docs/duvidas.md (tarefa 06).

Ao conectar no WebSocket, o cliente recebe as últimas 60 s antes do fluxo ao vivo, para o dashboard não abrir vazio. As mensagens usam um envelope (`{"type": "sample", "data": ...}` ou `{"type": "dtc", "data": ...}`).

4.8.2 Esquema do banco de dados

O banco tem cinco tabelas (Figura 6). A coluna `readings.true_value`, preenchida pelo executor de ensaios e nunca pelo coletor, é o que permite medir o erro de decodificação por uma consulta SQL, sem reler arquivos. Há uma restrição de unicidade em `samples(run_id, seq)`, que garante que uma amostra duplicada não seja gravada.

Figura 6: Modelo entidade-relacionamento do banco SQLite (colunas principais)



Fonte: docs/spec/07-dados-web.md (diagrama em docs/tcc/img/fig-er.mmd).

4.8.3 Dashboard

O dashboard (Figura 7), servido em `http://localhost:8000/`, recebe os dados por WebSocket e mostra: conta-giros com faixa vermelha em 6000 rpm, manômetro do coletor de admissão em bar relativo (com vácuo negativo), velocidade,

temperatura e combustível em mostradores numéricos, barras de tensão, carga, borboleta e ajuste de mistura, um gráfico de 60 s com séries selecionáveis, o estado da MIL com a lista de códigos de falha e uma lista de eventos de comunicação. O cabeçalho mostra o estado da conexão, a taxa (Hz), o ciclo (ms) e a latência. Há estados explícitos: conectado, reconectando e sem dados. Valores sem atualização por mais de 5 s ficam esmaecidos, para que um número antigo não seja confundido com um número atual.

Figura 7: Dashboard de telemetria com rotação sobreposta em 3000 rpm e o código P0300 armazenado

Fonte: captura de tela feita pelos autores (docs/img/dashboard.png).

4.8.4 Painel de controle do simulador

O painel de controle (Figura 8), na porta 8001, altera o simulador em tempo de execução: sliders para sobrepor rotação, velocidade, temperatura, borboleta, combustível e tensão (com botão para voltar ao automático), seletor de perfil de condução, chave “ECU sem resposta”, sliders de atraso, perda e corrupção de quadros, injeção e limpeza de DTCs e campos de semente com botão de reinício. Um monitor de quadros mostra, em hexadecimal, as requisições (azul) e as respostas (verde) trocadas com a ECU.

Figura 8: Painel de controle do simulador com monitor de quadros

Fonte: captura de tela feita pelos autores (docs/img/painel.png).

Os limites de entrada do painel foram escolhidos para evitar estados sem sentido: atraso de 0 a 1000 ms (acima disso o coletor, com timeout de 100 ms, já estourou), perda e corrupção de 0 a 1, sementes de 0 a $2^{32} - 1$, e cada variável aceita apenas a faixa que o respectivo PID consegue codificar; fora disso a API responde 422.

4.9 ESTRATÉGIA DE TESTES

A estratégia segue uma pirâmide de cinco níveis (Tabela 14). Os testes foram escritos antes da implementação em cada tarefa.

Tabela 14: Pirâmide de testes

Nível	O que cobre	Ferramenta
Unitário	Fórmulas de PID, DTC, quadros, ISO-TP, validação de resposta	pytest
Propriedade	decode(encode(x)) dentro de r/2 para 1001 valores por PID; remontagem ISO-TP para qualquer comprimento de 8 a 4095 bytes	hypothesis
Integração	Coletor contra a ECU pelo can_virtual; servidor com armazenamento em memória e em SQLite; servidor ELM327 com cliente falso e com cliente real	pytest-asyncio
Ponta a ponta	Simulador, coletor, servidor e WebSocket no ar; páginas do dashboard	pytest e subprocessos
Desempenho	Ensaios E1 a E7 com coleta de métricas	scripts em tests/benchmarks

Fonte: docs/spec/08-testes.md.

Os **dublês** de teste ficam em `tests/fakes.py`: `FakeClock` (tempo determinístico), `LoopbackTransport` e `ScriptedTransport` (respostas de uma lista fixa, para casos de borda e quadros inválidos), `FakeVehicleModel`, `InMemoryStorage`, `RecordingPublisher`, `FakeElm327Client` e `FakeBroker` (MQTT em memória). Dois testes ficam fora da execução padrão por exigirem infraestrutura externa: os marcados `broker` (precisam de um Mosquitto real) e `realclient` (instalam o python-OBD em um ambiente virtual temporário, o que exige rede). O python-OBD é instalado sempre em um ambiente separado porque o pacote do PyPI tem o mesmo nome (`obd`) do pacote deste projeto e o encobriria.

4.10 ENSAIOS

Sete ensaios medem a plataforma (Tabela 15). Em todos, o simulador e o coletor rodam como tarefas de um mesmo processo, com um servidor Uvicorn real em `127.0.0.1` e SQLite em arquivo; o coletor usa o `LoopbackTransport`. Isso significa que os ensaios não passam pelo `python-can` nem medem o custo do barramento virtual. O campo `source` das amostras diz `can_virtual` por herança de projeto, mas o caminho é o do laço local.

Tabela 15: Desenho dos ensaios

Ensaio	Cenário	Medidas
E1	240 s, 10 PIDs, ECU em 10 ± 3 ms, falha P0300 aos 100 s	Erro por PID, RTT, ciclo, latência, perdas, tempo de detecção da falha
E2	4 execuções de 30 s com a ECU em 0, 10, 25 e 50 ms	Ciclo e taxa em função do atraso
E3	4 execuções de 20 s com 1, 3, 5 e 10 PIDs	Taxa por PID e leituras por segundo
E4	4 execuções de 30 s com perda de 0, 1, 5 e 10% dos quadros	Timeouts, recuperação, taxa efetiva

Ensaio	Cenário	Medidas
E5	VIN por ISO-TP com STmin de 0, 1 e 10 ms, 100 repetições	Tempo de remontagem
E6	Cliente ELM327 de soquete cru contra o servidor TCP	Compatibilidade de 19 comandos
E7	2 execuções de 60 s, uma por HTTP e outra por MQTT	Latência e perdas por canal

Fonte: docs/spec/08-testes.md.

Métricas. As estatísticas reportadas são média, desvio padrão amostral (com n − 1), mediana, percentis 95 e 99 (pelo método do posto mais próximo) e máximo. Os requisitos RNF01 e RNF02 são verificados pelo p95, que é mais exigente que a média. O RNF03 é verificado como (publicadas menos recebidas) mais as lacunas do número de sequência. O RNF04 é verificado pelos identificadores de 11 bits e pelo RTT máximo menor que o P2 de 50 ms; os 500 kbit/s são nominais, já que o barramento virtual não tem taxa, e não são medidos.

Erro de decodificação. O erro é |x_dec − x_real|, comparado com metade da resolução do PID. Para o PID da rotação, a resolução é 0,25 rpm e o limite é 0,125 rpm. A decodificação só é considerada correta se o erro estiver dentro do limite em 100% das leituras.

Falha injetada. No E1, o código P0300 é agendado em 100/240 da duração (100 s no ensaio completo). O tempo de detecção é o instante em que o coletor publica o relatório com o código, menos o instante agendado. O piso teórico é o período de 2 s da consulta ao PID `0x01`.

Ambiente de medição. Todas as medições vêm de um único computador. Os arquivos JSON de cada ensaio registram a máquina: contêiner Linux sobre WSL2 (`Linux-6.6.87.2-microsoft-standard-WSL2-x86_64-with-glibc2.36`), Python 3.12.15, 32 núcleos lógicos. Os tempos são de relógio real; portanto dependem da carga do computador no momento. As execuções de E1 a E6 foram feitas em 24/10/2026 e a de E7 com Mosquitto real, na mesma data. Valores físicos reais e erros de decodificação dependem apenas das sementes (modelo 1, ECU 1) e do perfil; ciclo, RTT e latência dependem do sistema operacional e da CPU.

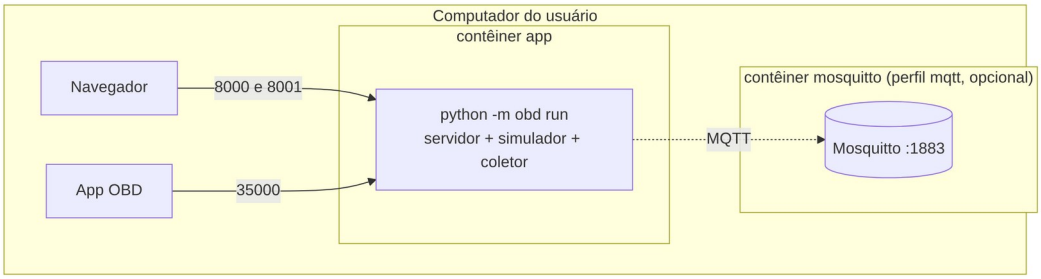
Repetição. A repetição dos ensaios em uma segunda máquina, prevista originalmente no RNF05, não foi realizada. As instruções para repeti-la, inclusive dentro de um contêiner Docker isolado, estão no Apêndice B. Uma repetição em contêiner na mesma máquina verificaria o isolamento e a facilidade de instalação,

mas não substituiria uma segunda máquina, e seria descrita como repetição em ambiente isolado, e não como reprodução independente.

4.11 AMBIENTE DE EXECUÇÃO E DOCKER

Para atender ao RNF07 e à restrição de não instalar ferramentas no computador do desenvolvedor, a plataforma é empacotada com Docker Compose (Figura 9). O serviço `app` executa `python -m obd run` e publica as portas 8000, 8001 e 35000. O serviço `mosquitto` usa a imagem `eclipse-mosquitto:2` e só sobe com o perfil `mqtt`. Um volume (`obd-data`) preserva o banco entre execuções. Um terceiro contêiner, de uso exclusivo do desenvolvimento, executa as tarefas noturnas descritas na Seção 4.12.

Figura 9: Implantação com Docker Compose



Fonte: elaborada pelos autores a partir de docker-compose.yml.

4.12 PROCESSO DE DESENVOLVIMENTO

O desenvolvimento foi organizado em tarefas pequenas (Tabela 16), cada uma descrita em um arquivo de `tasks/` com contexto, entregáveis, testes a escrever primeiro, critério de pronto e o modelo de linguagem de menor custo capaz de executá-la. Uma tarefa corresponde a uma sessão de trabalho, a um ramo do Git (`task/NN-nome`) e a um pedido de alteração revisado e integrado ao ramo principal. Nada é confirmado diretamente no ramo principal.

Tabela 16: Tarefas do desenvolvimento da primeira etapa

Tarefa	Conteúdo
00	Fundação: estrutura, configuração, Makefile, qualidade
01	Protocolo base: PIDs, DTCs, quadros e serviços
02	ISO-TP e modos 02 e 09
03	Transporte e ECU virtual
04	Modelo de veículo, perfis e falhas
05	Coletor
06	Servidor, API e banco

Tarefa	Conteúdo
07	Dashboard
08	Painel de controle do simulador
09	Camada ELM327 (servidor e cliente)
10	Ensaios E1 a E7
12	MQTT
13	Orquestração de make run e Docker
14	Correções do dashboard e do painel
15	Redesenho visual
16	Compatibilidade do ELM327 com clientes reais

Fonte: tasks/ e docs/PLAN.md.

Dois recursos de processo merecem registro. O primeiro é a **execução noturna**: um executor em Python (`scripts/run_night.py`) lê as tarefas, respeita as dependências e chama o assistente de código para cada uma, com um portão de qualidade (`make lint && make test`) que precisa passar antes de a tarefa ser marcada como concluída; o estado fica em `runs-night/state.json` e a execução é isolada em um contêiner. O segundo é o **registro de decisões** (`docs/duvidas.md`): cada ponto que a especificação não definia, a decisão tomada e o motivo, tarefa a tarefa, o que deixa explícitas as escolhas que um revisor poderia questionar.

O código e os textos deste trabalho foram produzidos com apoio de um assistente de inteligência artificial (Claude, da Anthropic), sob a definição de objetivos, a revisão e a validação dos autores. Os termos exatos dessa declaração constam na página própria, antes do sumário.

5 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados dos testes automatizados, dos sete ensaios de desempenho e da validação com clientes ELM327 de terceiros. Todos os números vêm de uma única máquina (Seção 4.10) e descrevem o simulador e a plataforma que o cerca. Nenhum deles descreve um veículo real. A origem de cada número está no Apêndice A.

5.1 TESTES AUTOMATIZADOS E COBERTURA

A suíte completa tem 405 testes e todos passaram na execução de 24/10/2026 (em Python 3.13.17, ambiente de revisão; a plataforma exige 3.12 ou superior e os ensaios rodaram em 3.12.15), com 2 testes excluídos da execução padrão por exigirem infraestrutura externa (`broker` e `realclient`, descritos na Seção 4.9). A cobertura de linhas, medida com `pytest-cov`, é de 89% no total (2311 linhas executáveis, 254 não cobertas). A Tabela 17 mostra a cobertura dos módulos de protocolo, que é a meta do requisito RNF06 (80% ou mais).

Tabela 17: Cobertura de testes nos módulos de protocolo

Arquivo	Linhas	Não cobertas	Cobertura
protocol/dtc.py	19	0	100%
protocol/errors.py	1	0	100%
protocol/frames.py	31	0	100%
protocol/isotp.py	166	1	99%
protocol/pids.py	64	0	100%
protocol/schemas.py	38	0	100%
protocol/services.py	122	4	97%

Fonte: saída de `pytest --cov=obd`, em 24/10/2026.

Os módulos de protocolo ficam entre 97% e 100%, bem acima da meta. Nos demais, a cobertura é alta (coletor 93% a 100%, servidor 89% a 100%, simulador 93% a 99%), com duas ressalvas. Os arquivos `runner.py` e `__main__.py` aparecem com 0% porque são exercitados por testes que os executam em subprocesso (inclusive o encerramento limpo por SIGINT e SIGTERM), e a medição de cobertura não acompanha o subprocesso. O módulo `mqtt_common.py` tem 51%, pois parte dele é o adaptador que só roda contra um *broker* real e é coberta apenas pelo teste `broker`, fora da execução padrão.

A suíte também inclui testes de propriedade com `hypothesis`: para cada PID, a decodificação do valor codificado volta ao original dentro de metade da resolução em 1001 valores da faixa, e a remontagem ISO-TP é correta para qualquer comprimento entre 8 e 4095 bytes. Esses testes cobrem o requisito RF06 no nível da função, enquanto o ensaio E1 o cobre no nível do sistema.

5.2 ENSAIO E1: EXATIDÃO E DESEMPENHO BASE

O E1 é o ensaio central: 240 s, 10 PIDs, ECU com atraso aleatório uniforme de 7 a 13 ms (10 ± 3 ms), perfil urbano, sementes 1 e 1, e uma falha P0300 agendada em 100 s. O canal de publicação é HTTP e o intervalo alvo do ciclo é de 100 ms. A Tabela 18 reúne os resultados.

Tabela 18: Resultados do ensaio E1 (240 s, 10 PIDs)

Medida	Resultado
Ciclos / leituras	1907 / 19070
Taxa	7,94 ciclos/s; 79,4 leituras/s
Ciclo (ms)	média 106,1; desvio 5,6; p95 115,6; p99 119,3; máx 124,0
RTT por PID (ms)	média 10,6; desvio 1,8; p95 13,5; p99 13,7; máx 14,1
Latência coletor ao banco (ms)	média 2,9; p95 3,5; p99 3,7; máx 42,2
Perdas, duplicações e timeouts	0, 0 e 0
Leituras com erro acima de r/2	0 de 19070
Detecção da P0300	0,79 s após a injeção

Fonte: docs/ensaios/e1.json.

O ciclo de 10 PIDs custa, em média, 106,1 ms, ou seja, cerca de 10 vezes o RTT médio por PID (10,6 ms). Como o atraso da ECU simulada é de 10 ms em média, o excedente de cerca de 0,6 ms por requisição é o custo de processamento do coletor, do transporte e da ECU somados. Como o ciclo (106 ms) é maior que o intervalo alvo (100 ms), o coletor não espera entre ciclos, e a taxa fica em 7,9 ciclos/s, não em 10.

A latência do coletor ao banco tem média de 2,9 ms e p95 de 3,5 ms. O máximo (42,2 ms) é um valor isolado: o p99 é 3,7 ms, o que indica um único evento ou poucos. Sua causa não foi investigada.

5.2.1 Exatidão da decodificação

Nenhuma das 19070 leituras teve erro maior que metade da resolução. A Tabela 19 detalha o erro por PID. Os erros máximos ficam muito próximos do limite

teórico, e isso é o esperado: o limite é o pior caso do arredondamento, e a distribuição dos valores reais cobre todo o passo de quantização. Se os erros máximos estivessem muito abaixo do limite, seria sinal de que os valores reais caem sempre sobre valores representáveis, ou seja, de um teste pouco exigente. O PID de tensão (0x42) tem resolução de 0,001 V e erro desprezível, porque o valor é transmitido em milivolts; o erro máximo ficou abaixo de 0,0005 V (arredondado para 0,000 na tabela).

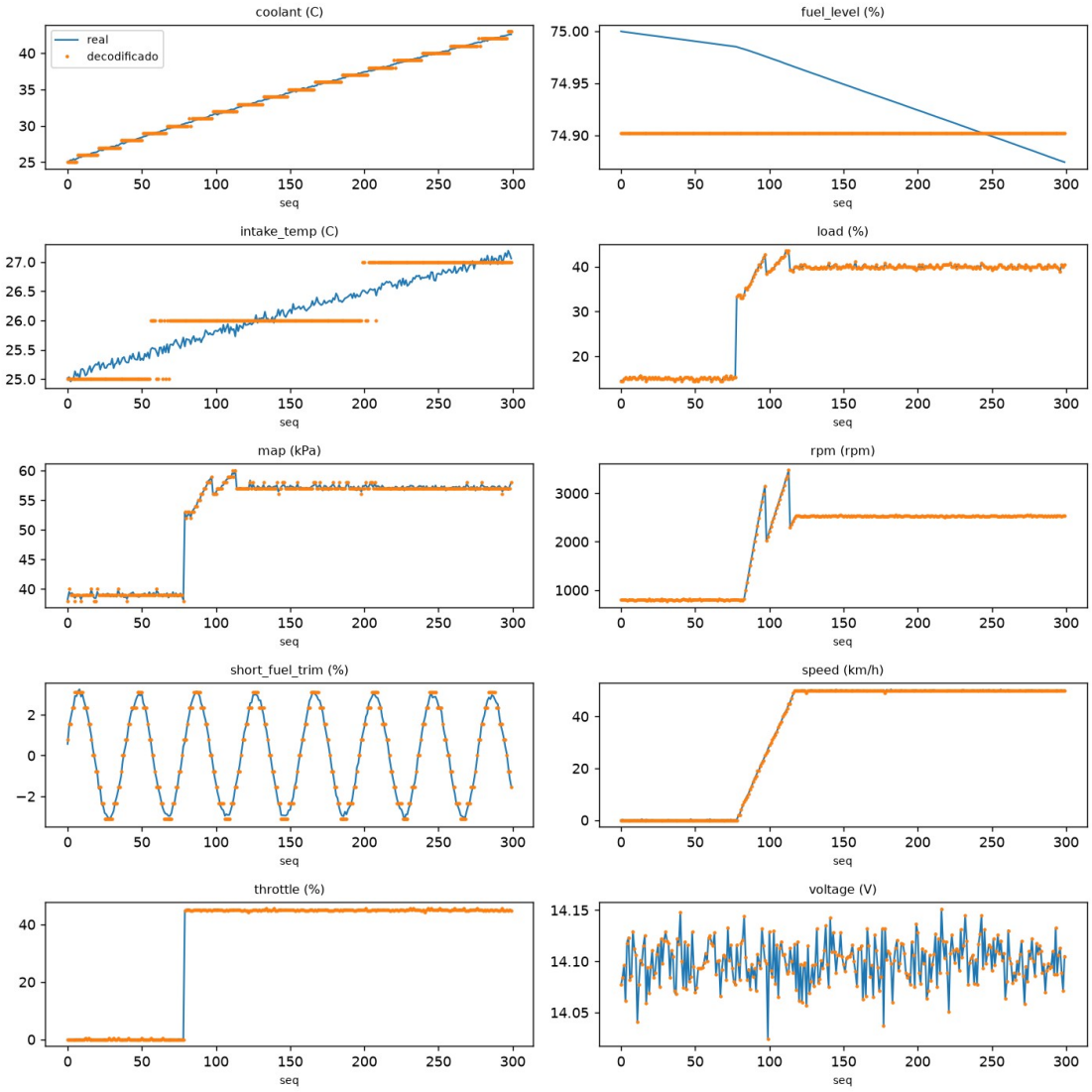
Tabela 19: Erro de decodificação por PID no ensaio E1

PID	Grandeza	Resolução	Leituras	Erro médio	Erro máx.	Limite r/2
0x04	Carga	0,392	1907	0,098	0,196	0,196
0x05	Temp. arrefecimento	1,000	1907	0,248	0,500	0,500
0x06	Ajuste de mistura	0,781	1907	0,199	0,390	0,391
0x0B	MAP	1,000	1907	0,231	0,500	0,500
0x0C	Rotação	0,250	1907	0,063	0,125	0,125
0x0D	Velocidade	1,000	1907	0,162	0,499	0,500
0x0F	Temp. ar de admissão	1,000	1907	0,264	0,500	0,500
0x11	Borboleta	0,392	1907	0,088	0,196	0,196
0x2F	Nível de combustível	0,392	1907	0,098	0,196	0,196
0x42	Tensão	0,001	1907	0,000	0,000	0,001

Fonte: docs/ensaios/e1.json (campo `por_pid`). Erros e limites na unidade de cada PID.

A Figura 10 mostra, para os 300 primeiros ciclos, o valor real (linha) e o decodificado (pontos) em cada PID. O efeito da quantização é visível em escala fina: na temperatura do ar de admissão, o valor real sobe de forma contínua e o decodificado avança em degraus de 1 °C; no nível de combustível, o valor real varia menos de 0,2 pontos percentuais e o decodificado fica em um único valor, dentro do limite. Na rotação, na velocidade e no ajuste de mistura, os dois acompanham a mesma forma.

Figura 10: Valor real e valor decodificado nos 10 PIDs (primeiros 300 ciclos do E1)



Fonte: docs/ensaios/e1_real_vs_decodificado.png, gerado de docs/ensaios/e1_amostra.csv.

5.2.2 Detecção da falha

A falha P0300 foi agendada em 100 s e o coletor publicou o relatório com o código 0,79 s depois. O coletor consulta o PID `0x01` a cada 2 s e, se o contador mudou, lê o modo 03. Com esse período, o atraso de detecção esperado varia entre perto de zero e 2 s, dependendo de em que ponto do período a falha cai, e 0,79 s está dentro dessa faixa. O requisito RF03 pedia detecção em até 2 ciclos de consulta (4 s): foi atendido.

5.2.3 Requisitos RNF01 a RNF04

A Tabela 20 compara o medido com o requisito.

Tabela 20: Requisitos não funcionais no ensaio E1

Requisito	Limite	Medido	Resultado
RNF01: ciclo de 10 PIDs (p95)	≤ 150 ms	115,6 ms	atendido
RNF02: latência do coletor ao banco (p95)	≤ 50 ms	3,5 ms	atendido
RNF03: perdas em 240 s	0	0	atendido
RNF04: identificadores de 11 bits (0x7DF e 0x7E8) e RTT ≤ P2	≤ 50 ms	RTT máx. 14,1 ms	atendido; 500 kbit/s não é medido

Fonte: docs/ensaios/e1.json (campo `rnf`).

O atendimento do RNF04 é parcial por construção. Os identificadores são os da norma e o RTT máximo é bem menor que o P2 de 50 ms, mas a taxa de 500 kbit/s é um parâmetro nominal: o barramento virtual não tem camada física, e portanto não há o que medir.

5.3 ENSAIO E2: SENSIBILIDADE AO ATRASO DA ECU

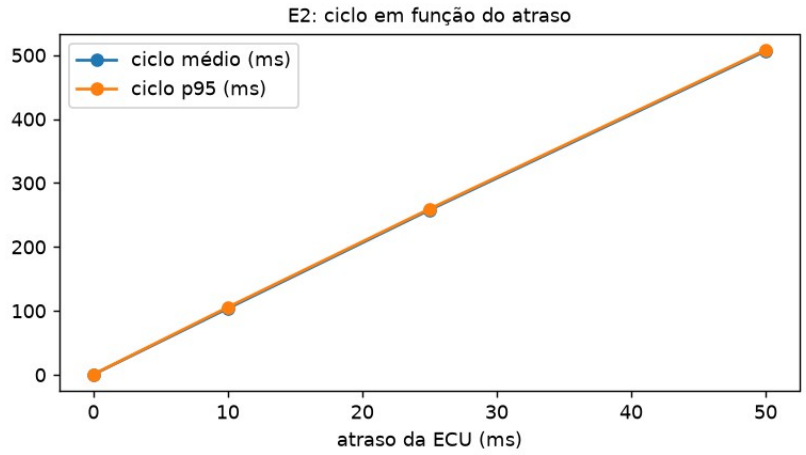
O E2 repete o cenário com o atraso da ECU fixo em 0, 10, 25 e 50 ms, 30 s cada (Tabela 21 e Figura 11).

Tabela 21: Resultados do ensaio E2

Atraso	Ciclo médio (ms)	Ciclo p95 (ms)	Ciclos/s	Leituras/s	RNF01
0 ms	0,5	0,7	9,9	99,4	atendido
10 ms	103,8	105,8	8,1	81,1	atendido
25 ms	257,8	259,5	3,5	35,4	não atendido
50 ms	506,4	508,2	1,8	18,5	não atendido

Fonte: docs/ensaios/e2.json; docs/ensaios.md.

Figura 11: Tempo de ciclo em função do atraso da ECU (E2)



Fonte: docs/ensaios/e2_sensibilidade.png.

O ciclo cresce de forma linear, em cerca de 10 vezes o atraso, o que reflete a natureza sequencial do coletor: cada um dos 10 PIDs é requisitado depois que o

anterior respondeu. Com atraso de 0 ms, o ciclo é menor que 1 ms e a taxa fica em 10 ciclos/s, limitada pelo intervalo alvo de 100 ms e não pelo custo de processamento. O limite do RNF01 (150 ms) é ultrapassado a partir de aproximadamente 14 ms de atraso por PID, e com 25 ms ou mais ele não é atendido. Isso não é um defeito da plataforma, e sim uma propriedade do modo de requisição serial: o RNF01 foi definido para uma ECU em 10 ms, e o E2 mostra até onde essa premissa se sustenta.

O ensaio com 50 ms também ultrapassa o RTT máximo de 50 ms do RNF04 (RTT máximo de 51,2 ms), o que é coerente: a ECU responde com esse atraso fixo, mais o processamento do coletor. Quando isso acontece, uma ECU real excederia o P2, e uma ferramenta que respeitasse a norma trataria o PID como sem resposta. O coletor desta plataforma usa um *timeout* de 100 ms, mais folgado, e por isso ainda lê normalmente.

Uma estratégia para recuperar o requisito com ECUs mais lentas seria requisitar mais de um PID por mensagem (a SAE J1979 prevê, em CAN, mais de um PID por requisição do modo 01) ou paralelizar as requisições; nenhuma das duas está implementada, e ficam como melhoria futura.

5.4 ENSAIO E3: NÚMERO DE PIDS

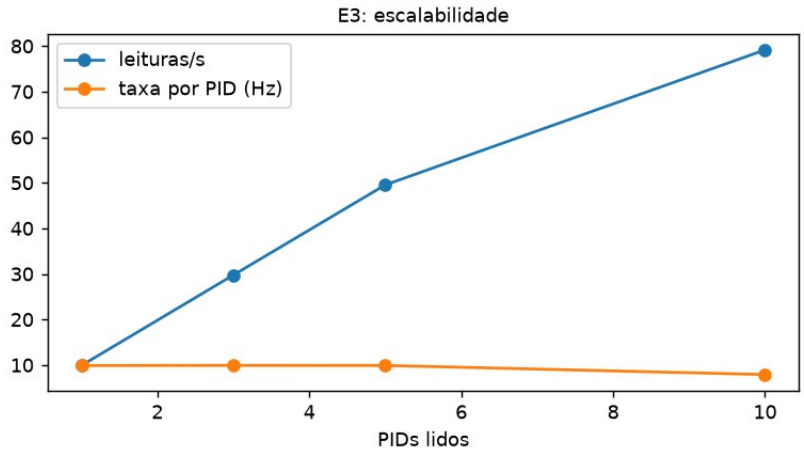
O E3 varia o número de PIDs por ciclo (1, 3, 5 e 10), com a ECU em 10 ± 3 ms e 20 s por execução (Tabela 22 e Figura 12).

Tabela 22: Resultados do ensaio E3

PIDs	Ciclo médio (ms)	Ciclo p95 (ms)	Taxa por PID (Hz)	Leituras/s
1	10,6	13,6	9,9	9,9
3	32,0	37,3	9,9	29,7
5	53,5	59,7	9,9	49,5
10	106,6	116,9	7,9	79,2

Fonte: docs/ensaios/e3.json; docs/ensaios.md.

Figura 12: Taxa por PID e leituras por segundo em função do número de PIDs (E3)



Fonte: docs/ensaios/e3_escalabilidade.png.

O ciclo cresce linearmente com o número de PIDs, em cerca de 10,6 ms por PID, igual ao RTT médio (10,6 ms em todas as execuções, o que mostra que o custo marginal de um PID extra é exatamente o de uma requisição). Até 5 PIDs, o ciclo cabe dentro do intervalo alvo de 100 ms e cada PID é lido a 9,9 Hz. Com 10 PIDs, o ciclo ultrapassa o intervalo e a taxa por PID cai para 7,9 Hz. A vazão total, em leituras por segundo, continua crescendo (de 9,9 a 79,2), mas a taxa de atualização de cada PID deixa de ser a desejada. O plano de leitura com PIDs rápidos e lentos, descrito na Seção 4.7, existe justamente para isso: ele reserva o ciclo curto para as grandezas que variam depressa (rotação, MAP, borboleta, velocidade) e lê as demais a cada 5 ciclos.

5.5 ENSAIO E4: PERDA DE QUADROS

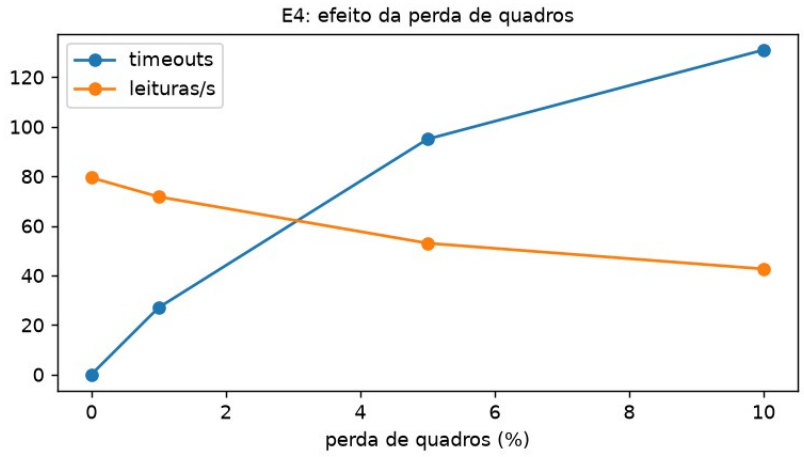
O E4 injeta perda de 0, 1, 5 e 10% das requisições recebidas pela ECU, com 10 PIDs e 30 s por execução (Tabela 23 e Figura 13). Para o coletor, uma resposta perdida é um *timeout* de 100 ms.

Tabela 23: Resultados do ensaio E4

Perda	Ciclos	Timeouts	Timeouts por ciclo	Ciclo médio (ms)	Leituras/s	Máx. de ciclos seguidos sem leitura de um PID
0%	239	0	0,00	106,3	79,4	0
1%	218	27	0,12	117,3	71,7	1
5%	169	95	0,56	157,0	53,0	2
10%	141	131	0,93	190,5	42,6	2

Fonte: docs/ensaios/e4.json; docs/ensaios.md.

Figura 13: Timeouts e leituras por segundo em função da perda de quadros (E4)



Fonte: docs/ensaios/e4_perdas.png.

O custo da perda é, em primeira aproximação, o do *timeout*: cada resposta perdida acrescenta cerca de 100 ms ao ciclo. Com 1% de perda, há em média 0,12 timeout por ciclo e o ciclo médio sobe 11 ms (de 106,3 para 117,3), compatível com 0,12 × 100 ms. Com 5%, são 0,56 timeouts por ciclo e o ciclo sobe 51 ms, próximo dos 56 ms que o mesmo cálculo daria. Com 10%, o ciclo sobe 84 ms, um pouco menos que os 93 ms que o mesmo raciocínio daria, o que sugere que algum efeito adicional (como o recuo progressivo, que age após três falhas seguidas) interfere; essa diferença não foi investigada.

O ponto mais importante é a recuperação. Em nenhuma execução houve mais de 2 ciclos seguidos sem leitura de um mesmo PID, o servidor nunca perdeu uma amostra (publicadas e recebidas são iguais em todos os casos) e o coletor sempre seguiu para o PID seguinte depois do *timeout*. A perda reduz a vazão (de 79,4 para 42,6 leituras por segundo com 10%), mas não corrompe nem interrompe a cadeia. Esse é o comportamento esperado pelo requisito RF11.

5.6 ENSAIO E5: REMONTAGEM ISO-TP

O E5 mede o tempo para ler o VIN (modo 09, PID 02), que exige um primeiro quadro, um controle de fluxo e dois quadros consecutivos, com STmin de 0, 1 e 10 ms, em 100 repetições cada (Tabela 24 e Figura 14).

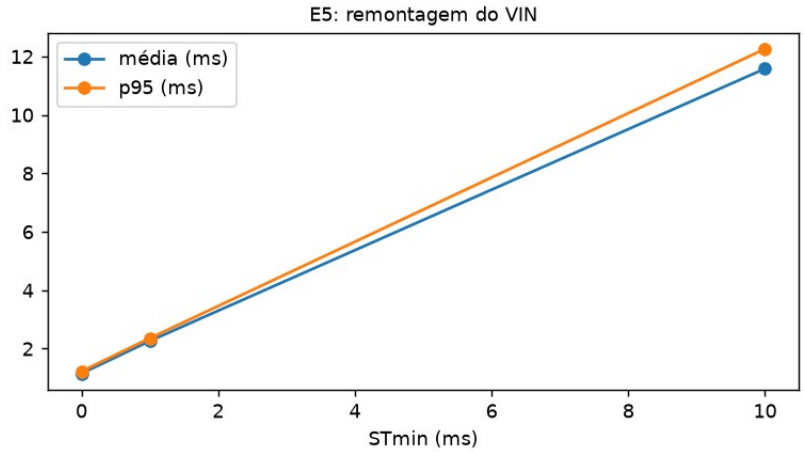
Tabela 24: Tempo de remontagem do VIN (E5, 100 repetições)

STmin	Média (ms)	Desvio	Mediana	p95	p99	Máx
0 ms	1,15	0,04	1,14	1,22	1,23	1,24
1 ms	2,26	0,06	2,25	2,36	2,45	2,47

STmin	Média (ms)	Desvio	Mediana	p95	p99	Máx
10 ms	11,58	0,45	11,35	12,26	12,29	12,31

Fonte: docs/ensaios/e5.json; docs/ensaios.md.

Figura 14: Tempo de remontagem do VIN em função do STmin (E5)



Fonte: docs/ensaios/e5_stmin.png.

O tempo de remontagem é aproximadamente o custo base de 1,15 ms somado ao STmin: 2,26 ms com 1 ms e 11,58 ms com 10 ms, o que é compatível com a espera de um intervalo STmin entre os dois quadros consecutivos. Em todas as 300 repetições, o VIN remontado estava correto. Com STmin de 10 ms, o desvio padrão é maior (0,45 ms), como esperado de uma espera por temporizador. Para o uso do coletor, a leitura do VIN ocorre uma vez, e mesmo com STmin de 10 ms o custo (cerca de 12 ms) cabe dentro do P2 de 50 ms.

5.7 VALIDAÇÃO COM CLIENTES ELM327 DE TERCEIROS

A camada ELM327 foi validada em três níveis crescentes de independência: um cliente de soquete cru escrito pelos autores (E6), a biblioteca python-OBD e o aplicativo OBD Auto Doctor.

5.7.1 E6: cliente de soquete cru

O E6 envia 19 comandos ao servidor TCP e confere cada resposta com uma expressão regular: `ATZ`, `ATE0`, `ATL0`, `ATS1`, `ATH0`, `ATSP0`, `ATDP`, `ATRV`, `0100`, `0120`, `010C`, `010D`, `0105`, `0111`, `0101`, `03`, `0902`, `01FF` e `XYZ`. As 19 respostas vieram no formato esperado. O E6 comprova que o servidor cumpre a especificação que os próprios autores escreveram, mas não que essa especificação reproduz o que um cliente real espera. Esse é o papel dos dois níveis seguintes.

5.7.2 python-OBD

A biblioteca python-OBD, na versão 0.7.3, foi apontada para o servidor (`socket://127.0.0.1:<porta>`) pelo teste de integração `test_elm327_python_obd.py`, que instala a biblioteca em um ambiente virtual temporário. A primeira execução, antes das correções da tarefa 16, conectou e reconheceu o protocolo (`ISO 15765-4 (CAN 11/500)`), mas não leu nenhum PID. A análise da troca de comandos mostrou três causas:

1. o servidor ignorava `ATH1`, e a biblioteca descartava os quadros sem cabeçalho (“dropping frame for being odd”);
2. os comandos `ATDPN` e `ATTP6` recebiam `?`, quando deveriam responder `A6` e `OK`;
3. os modos `0900` e `0600` recebiam `7F ...`, quando um adaptador real responderia a máscara de PIDs do modo 09 e `NO DATA`, respectivamente.

Depois da correção, a biblioteca reporta “Car Connected”, o protocolo correto, e lê rotação, velocidade, temperatura do líquido de arrefecimento, carga, borboleta e nível de combustível, todos dentro da resolução do PID. Com a rotação sobreposta em 3000 rpm no simulador, a biblioteca lê 3000. Com o P0300 injetado, `GET_DTC` devolve P0300, e sem falhas devolve lista vazia.

O resultado ilustra bem o valor de um cliente independente. A suíte de testes passava, porque era escrita contra a própria interpretação dos autores sobre o dialeto. Só um terceiro mostrou que a interpretação estava incompleta.

5.7.3 OBD Auto Doctor

O aplicativo OBD Auto Doctor, em versão de avaliação para Windows, foi configurado com conexão Wi-Fi apontando para o computador que executa o simulador, na porta 35000, e protocolo automático. Na primeira tentativa, exibiu “Failed to initialize the adapter”. A causa foi o dialeto ainda incompleto: o aplicativo enviava `ATAT1`, `ATST62` e `ATDPN` (recebendo `?`) e consultava `ATI`, que também não era reconhecido. Com as correções da tarefa 16 e o tratamento de `ATI` e `ATWS` (que passaram a devolver a versão do dispositivo), a conexão foi estabelecida.

Com o simulador em execução e a rotação sobreposta em 3000 rpm, os valores exibidos pelo aplicativo coincidiram com os do painel de controle do simulador. A tela de códigos de falha listou P0300 e P0301, injetados pelo painel, e a limpeza pelo aplicativo (modo 04) zerou a lista. Essas observações foram feitas

manualmente pelos autores, em uma única sessão; não há medida estatística associada, e o aplicativo, por ser de terceiros e de versão de avaliação, pode se comportar de forma diferente em outras versões. Os demais aplicativos citados no README (Car Scanner e Torque) não foram validados.

Para facilitar o diagnóstico de futuros clientes, o servidor registra cada comando recebido e a resposta enviada quando a variável de ambiente `OBD_ELM327_LOG=1` está ativa. Foi o que permitiu identificar `ATI` como a causa da falha do OBD Auto Doctor.

5.8 ENSAIO E7: HTTP CONTRA MQTT

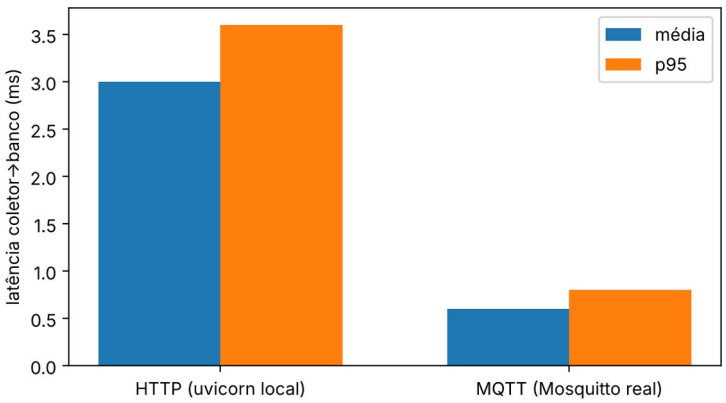
O E7 compara os dois canais de publicação, 60 s cada, com 10 PIDs, atraso da ECU de 7 a 13 ms e as mesmas sementes. A execução com MQTT usou um Mosquitto real (imagem `eclipse-mosquitto:2`), em contêiner no mesmo computador, e o simulador e o coletor em outro contêiner, pela rede interna do Docker. Os dois canais rodaram em sequência na mesma máquina (Tabela 25 e Figura 15).

Tabela 25: Resultados do ensaio E7 (60 s por canal, Mosquitto real)

Canal	Ciclos	Latência média (ms)	p95 (ms)	Máx (ms)	Perdas	Ciclos/s
HTTP (Uvicorn local)	463	3,0	3,6	43,5	0	7,7
MQTT (Mosquitto real)	558	0,6	0,8	13,4	0	9,3

Fonte: docs/ensaios/e7_mosquitto.json; docs/ensaios.md.

Figura 15: Latência do coletor ao banco por canal de publicação (E7, Mosquitto real)



Fonte: elaborada pelos autores a partir de docs/ensaios/e7_mosquitto.json (médias e p95 da Tabela 25).

Os dois canais entregaram todas as amostras: perdas, duplicações e *timeouts* foram zero, e o erro de decodificação acima de meia resolução também. O MQTT teve latência média cerca de cinco vezes menor (0,6 ms contra 3,0 ms) e máximo

cerca de três vezes menor. O tempo de ciclo do coletor e o RTT da ECU foram praticamente iguais (ciclo p95 de 115,8 ms no HTTP e 115,7 ms no MQTT; RTT p95 de 13,5 ms nos dois), pois dependem da ECU simulada e não do canal.

A diferença em ciclos por segundo (7,7 contra 9,3) vem do tempo gasto publicando: no HTTP, o coletor espera a resposta de cada envio antes de seguir; no MQTT com QoS 0, ele entrega a mensagem ao cliente e continua. Essa explicação é coerente com o comportamento dos protocolos, mas não foi verificada por uma medição específica.

As condições limitam a interpretação. Publicador e *broker* estavam no mesmo computador, por uma rede interna do Docker, e o servidor HTTP também era local. Os números descrevem o custo de cada canal nesse arranjo e não representam um enlace real entre máquinas diferentes, em que a latência de rede e a perda de pacotes alterariam as duas curvas. Além disso, a execução MQTT usou QoS 0, o que explica parte da vantagem (não há confirmação de recebimento) e também o risco: em caso de queda do *broker*, uma mensagem em trânsito pode se perder, e o servidor a vê como lacuna do número de sequência.

Um arquivo anterior do E7 (`docs/ensaios/e7.json`) foi gerado sem *broker* real, com um intermediário em memória usado nos testes. Nele, a latência MQTT média é 0,2 ms, que exclui rede e *broker*. Esse arquivo é mantido apenas como exemplo do formato, e os números deste trabalho são os do Mosquitto real.

5.9 REPRODUTIBILIDADE E AMEAÇAS À VALIDADE

Reprodutibilidade determinística. Com as mesmas sementes e o mesmo perfil, a sequência de valores físicos do simulador é idêntica, e portanto os valores reais, os valores decodificados e os erros de decodificação coincidem entre execuções. Esse resultado é consequência do projeto (sementes explícitas, relógio injetável e passos fixos de 10 ms) e é verificado pelos testes do modelo de veículo.

Reprodutibilidade de tempos. Ciclo, RTT e latência dependem de relógio real, do sistema operacional e da CPU, e não são reproduzidos bit a bit. Os números deste capítulo vêm de uma única execução de cada ensaio em uma única máquina (contêiner Linux sobre WSL2, 32 núcleos lógicos, Python 3.12.15). Como não há repetições independentes, não há como estimar a variação entre execuções; os desvios padrão reportados são dentro de uma execução. A repetição em uma

segunda máquina, prevista no requisito RNF05, não foi feita e é um trabalho pendente (Apêndice B).

Ameaças à validade. As principais são: (i) a ECU simulada é um modelo, e o que ela responde é o que os autores entenderam da norma; a validação com python-OBD e OBD Auto Doctor reduz esse risco, mas não o elimina; (ii) o barramento CAN não é reproduzido fisicamente, e os ensaios usam o laço local, sem custo de barramento; (iii) os tempos de relógio real foram medidos com outras tarefas rodando no mesmo computador, sem isolamento de CPU; (iv) o atraso da ECU é uma variável aleatória simples (uniforme), enquanto uma ECU real tem distribuição própria; (v) o modelo de veículo é ilustrativo, e as variáveis são coerentes entre si, mas não calibradas contra dados reais.

5.10 ATENDIMENTO AOS REQUISITOS

A Tabela 26 resume a situação de cada requisito e a evidência.

Tabela 26: Atendimento aos requisitos

Requisito	Situação	Evidência
RF01 a RF05 (protocolo, descoberta, DTCs, freeze frame, ISO-TP)	Atendidos	Testes unitários, de propriedade e de integração; E5 (VIN)
RF06 (decodificação com erro ≤ r/2)	Atendido	E1: 0 de 19070 leituras acima do limite
RF07 (persistência sem perda nem duplicação)	Atendido	E1 a E7: perdas e duplicações iguais a 0
RF08 (API e WebSocket)	Atendido	Testes de integração e de ponta a ponta; dashboard
RF09 (painel de controle)	Atendido	Testes da API de controle; Figuras 7 e 8
RF10 (camada ELM327)	Atendido para python-OBD e OBD Auto Doctor	Seção 5.7
RF11 (falhas de comunicação)	Atendido	E4; testes de recuo e de ECU muda
RF12 (transporte plugável)	Atendido para loopback, can_virtual e elm327_tcp	Testes de integração dos três transportes
RNF01 a RNF03	Atendidos	Tabela 20
RNF04 (conformidade ISO 15765-4)	Parcial	Identificadores e P2 verificados; 500 kbit/s não medido
RNF05 (reprodutibilidade)	Parcial	Determinismo verificado; segunda máquina pendente
RNF06 (cobertura ≥ 80% no protocolo)	Atendido	97% a 100% (Tabela 17)
RNF07 (instalação com um comando)	Atendido	make run e docker compose up app
RNF08 (somente leitura, sem exposição externa)	Atendido	Escuta em 127.0.0.1 por padrão; modo 04 só no simulador

Fonte: elaborada pelos autores.

5.11 DISCUSSÃO

Quatro observações emergem do conjunto de resultados.

A plataforma cumpre o que prometeu como ambiente de validação. Os requisitos de exatidão, latência e perda foram atendidos com folga (ciclo p95 de 115,6 ms contra 150 ms; latência p95 de 3,5 ms contra 50 ms; nenhuma perda), e a capacidade de medir o erro contra um valor real conhecido é o que distingue este trabalho de um simples leitor OBD-II. A margem, porém, não deve ser lida como desempenho em um veículo: o ambiente tem RTT de 10 ms por escolha do modelo e sem a temporização do barramento físico.

O gargalo é o modo de requisição serial, não o processamento. O custo de processamento é de cerca de 0,6 ms por requisição (E1, coletor, transporte e ECU somados), e quase todo o ciclo é espera pela ECU. Isso explica a linearidade do E2 e do E3, e aponta onde otimizar: requisitar vários PIDs por mensagem, ou reduzir o número de PIDs lidos a cada ciclo por meio do plano rápido e lento.

O comportamento sob falha é robusto, mas tem preço previsível. O E4 mostra que cada resposta perdida custa um *timeout* e que a vazão cai de forma quase proporcional, sem perda no servidor e sem travar o coletor. Para uma aplicação que precise de taxa mínima garantida, o *timeout* de 100 ms poderia ser reduzido (o P2 da norma é 50 ms), com o risco de tratar como perdidas respostas apenas lentas.

Validação por terceiros revela o que testes próprios não revelam. O ciclo de correções da camada ELM327 (Seção 5.7) é o exemplo mais claro: a suíte estava toda verde, e dois clientes independentes falharam na primeira tentativa. O ganho de ter um transporte intercambiável e uma camada de compatibilidade é justamente poder exercitar o software contra o mundo externo antes de ligar qualquer hardware.

6 CONSIDERAÇÕES FINAIS

Este trabalho propôs, implementou e validou uma plataforma de telemetria OBD-II inteiramente em software, com o objetivo de oferecer um ambiente em que a cadeia completa de coleta, armazenamento e visualização possa ser desenvolvida, medida e repetida sem hardware. Este capítulo retoma os objetivos, resume as contribuições, declara as limitações e descreve a segunda etapa do projeto.

6.1 CONCLUSÕES

O objetivo geral foi atingido no escopo definido. A plataforma reproduz a cadeia de telemetria OBD-II (veículo simulado, ECU virtual conforme a ISO 15765-4, transporte plugável, coletor, servidor com banco de dados, dashboard e painel de controle) e pode ser executada com um comando. Quanto aos objetivos específicos:

1. **Requisitos.** Doze requisitos funcionais e oito não funcionais foram especificados com critérios de aceitação. Dos oito não funcionais, seis foram atendidos e dois (RNF04 e RNF05) foram atendidos parcialmente, por motivos declarados: a taxa de 500 kbit/s não é medida em um barramento virtual, e a repetição em segunda máquina não foi feita.
2. **Protocolo.** O núcleo de protocolo implementa ISO-TP completo, os modos 01, 02, 03, 04, 07 e 09, dez PIDs, a descoberta de PIDs e a codificação de DTCs, sem entrada e saída, com 97% a 100% de cobertura de testes.
3. **Simulador.** O modelo de veículo e a ECU virtual são determinísticos, e o catálogo de falhas cobre DTCs, atraso, perda, corrupção, ECU muda e atraso de ISO-TP.
4. **Coletor, servidor e interfaces.** Foram implementados e integrados, e funcionam em HTTP e MQTT.
5. **Transporte plugável.** A interface `Transport` tem três implementações, e o coletor não muda entre elas.
6. **ELM327.** A camada de compatibilidade foi validada com a biblioteca python-OBD e com o aplicativo OBD Auto Doctor, depois de correções motivadas pela falha inicial dos dois.
7. **Medições.** Em 19070 leituras do ensaio E1, nenhuma teve erro de decodificação acima de metade da resolução; o ciclo de 10 PIDs teve p95 de

115,6 ms, a latência do coletor ao banco teve p95 de 3,5 ms, e não houve perdas. Os ensaios E2 a E5 mostraram que o ciclo cresce linearmente com o atraso da ECU e com o número de PIDs, que a perda de quadros reduz a vazão sem perda de dados no servidor, e que o tempo de remontagem do VIN é o custo base somado ao STmin. O E7 mostrou, com um *broker* Mosquitto real no mesmo computador, latência média de 0,6 ms no MQTT contra 3,0 ms no HTTP.

8. **Documentação.** As escolhas, as decisões e as limitações estão registradas no repositório, em especial em `docs/spec/` e `docs/duvidas.md`.

6.2 CONTRIBUIÇÕES

A principal contribuição é metodológica: um ambiente em que o valor real de cada grandeza é conhecido e gravado ao lado do valor decodificado, o que permite medir a exatidão do sistema em vez de supô-la. A segunda é a possibilidade de injetar falhas de comunicação e de aplicação de modo repetível, a partir de sementes numéricas. A terceira é a camada de compatibilidade ELM327, que transforma bibliotecas e aplicativos de terceiros em clientes de teste independentes, e cujo desenvolvimento mostrou que testes escritos pelos próprios autores deixaram passar divergências que clientes externos expuseram. A quarta é a interface de transporte, que deixa o ponto de integração com o hardware definido e estável.

Do ponto de vista de engenharia de software, o projeto demonstra um fluxo de trabalho em que a especificação precede o código, os testes precedem a implementação, as decisões não especificadas ficam registradas, e cada alteração entra por pedido de alteração revisado. O fluxo também incorpora um assistente de inteligência artificial como ferramenta, com os autores responsáveis pela definição dos objetivos, pela revisão e pela validação.

6.3 LIMITAÇÕES

As limitações abaixo delimitam o que os resultados permitem afirmar.

- **Nenhum veículo real foi usado.** Todos os resultados valem para o simulador e para a plataforma que o cerca. O trabalho não afirma nada sobre o comportamento de um veículo, de uma ECU real ou de um adaptador físico.

- **Barramento virtual.** Os ensaios usam o laço local, sem camada física CAN. Não há medida de taxa de bits, terminação, arbitragem ou erros de bit.
- **Uma única máquina.** Os tempos vêm de uma única execução de cada ensaio em um computador, sem repetições independentes. Os desvios padrão são internos a cada execução, e a variação entre execuções e entre máquinas não foi estimada.
- **ECU e veículo são modelos.** O atraso da ECU é uniforme e o modelo do veículo é ilustrativo e não calibrado. A norma foi interpretada pelos autores e validada contra dois clientes de terceiros, o que reduz, mas não elimina, o risco de divergência em relação a outros clientes e a uma ECU real.
- **Validação de aplicativos de celular.** Car Scanner e Torque não foram testados. O OBD Auto Doctor foi observado manualmente em uma sessão, sem medida estatística.
- **Segurança.** O sistema não usa TLS nem autenticação. A API de controle do simulador deve ser exposta apenas em rede local de confiança.
- **Escopo do protocolo.** Apenas CAN de 11 bits (ISO 15765-4), um conjunto de dez PIDs e seis modos de serviço. K-Line, J1850, CAN de 29 bits, o modo 06 e outros PIDs ficam fora.
- **Explicações não verificadas.** Duas explicações apresentadas no Capítulo 5 (a diferença de ciclos por segundo entre HTTP e MQTT, e o comportamento do ciclo com 10% de perda) são coerentes com o funcionamento do sistema, mas não foram confirmadas por medição específica, e o máximo de latência de 42,2 ms no E1 não foi investigado.

6.4 TRABALHOS FUTUROS

6.4.1 Segunda etapa: hardware

A segunda etapa do projeto, planejada e ainda não executada, substitui o simulador por hardware. A ideia é usar uma placa de desenvolvimento com o microcontrolador ESP32, com o periférico TWAI (a interface CAN do chip) e um transceptor CAN de 3,3 V como o SN65HVD230, ligado ao conector OBD-II do veículo pelos pinos de sinal CAN (6 e 14), terra (4 e 5) e, se for o caso, alimentação (16) (ESPRESSIF SYSTEMS, [s.d.]a; [s.d.]b; TEXAS INSTRUMENTS, [s.d.]; SAE INTERNATIONAL, 2016a). O firmware faria dois papéis: falar CAN com o veículo,

usando a mesma lógica do núcleo de protocolo deste trabalho, e expor ao exterior um servidor TCP no dialeto ELM327, por Wi-Fi. Com isso, o computador que executa a plataforma, ou um aplicativo de celular, conectaria ao dispositivo exatamente como se conecta ao simulador, pela porta 35000.

Algumas decisões já tomadas para a segunda etapa, que dependem de validação em campo:

- o dispositivo seria **somente de leitura**: o modo 04 ficaria desabilitado no firmware, em coerência com o RNF08 e com a literatura de segurança (KOSCHER et al., 2010; CHECKOWAY et al., 2011);
- como a rede CAN do veículo já tem as terminações próprias, a terminação de 120 Ω do módulo transceptor deveria ser removida ou desabilitada;
- o dispositivo teria dois modos de rede: ponto de acesso próprio (endereço fixo, como um adaptador comercial) ou cliente de uma rede existente, com descoberta por nome (mDNS);
- as tarefas 17, 18 e 19 do repositório descrevem, respectivamente, a ampliação da conformidade do servidor ELM327, a adaptação do coletor a um adaptador real e o firmware.

Antes de qualquer ligação ao veículo, o protocolo de diagnóstico suportado por ele precisa ser verificado, porque a plataforma implementa apenas CAN de 11 bits a 500 kbit/s, e veículos mais antigos podem usar outro protocolo. Qualquer resultado da segunda etapa deve ser tratado como um trabalho distinto, com metodologia e limitações próprias, e não como extensão automática das conclusões deste.

6.4.2 Melhorias da plataforma

- Repetir os ensaios em uma segunda máquina e com várias execuções por ensaio, para estimar a variação entre execuções.
- Requisitar mais de um PID por mensagem, para reduzir o ciclo com ECUs de maior atraso.
- Implementar a leitura dos modos 06 e 07 no coletor e de mais PIDs, incluindo os de consumo de combustível e de sensores de oxigênio.
- Adicionar TLS e autenticação ao canal HTTP e ao MQTT.

- Medir o canal MQTT em rede real entre máquinas, com QoS 1 e perdas de pacotes induzidas.
- Validar Car Scanner e Torque e documentar o resultado, e salvar as capturas de tela do OBD Auto Doctor no repositório.
- Construir um aplicativo móvel e considerar armazenamento especializado em séries temporais para volumes maiores de dados.
- Explorar análise dos dados coletados, como detecção de anomalias e manutenção preditiva.

6.5 CONSIDERAÇÃO FINAL

O trabalho mostra que boa parte da engenharia de um sistema de telemetria veicular pode ser feita, medida e repetida sem tocar em um carro, desde que o ambiente de simulação conheça a verdade que o sistema tenta medir. A distância entre esse ambiente e um veículo real é grande e foi mantida explícita ao longo do texto. A interface de transporte é o lugar onde essa distância será percorrida na segunda etapa.

REFERÊNCIAS

AL-FUQAHA, A. et al. Internet of Things: a survey on enabling technologies, protocols, and applications. IEEE Communications Surveys & Tutorials, v. 17, n. 4, p. 2347-2376, 2015.

ATZORI, L.; IERA, A.; MORABITO, G. The Internet of Things: a survey. Computer Networks, v. 54, n. 15, p. 2787-2805, 2010.

BOSCH, R. CAN Specification: version 2.0. Stuttgart: Robert Bosch GmbH, 1991.

BRASIL. Conselho Nacional do Meio Ambiente. Resolução CONAMA nº 354, de 13 de dezembro de 2004. Dispõe sobre os requisitos para adoção de sistemas OBD nos veículos automotores leves objetivando preservar a funcionalidade dos sistemas de controle de emissão. Brasília, DF, 2004.

CHECKOWAY, S. et al. Comprehensive experimental analyses of automotive attack surfaces. In: USENIX SECURITY SYMPOSIUM, 20., 2011, San Francisco. Proceedings […]. Berkeley: USENIX Association, 2011.

COPPOLA, R.; MORISIO, M. Connected car: technologies, issues, future trends. ACM Computing Surveys, v. 49, n. 3, art. 46, 2016.

DAVIS, R. I. et al. Controller Area Network (CAN) schedulability analysis: refuted, revisited and revised. Real-Time Systems, v. 35, n. 3, p. 239-272, 2007.

DENTON, T. Automobile electrical and electronic systems. 5. ed. Abingdon: Routledge, 2017.

DOCKER. Docker Compose overview. [S.l.]: Docker, [s.d.]. Disponível em: https://docs.docker.com/compose. Acesso em: 24 out. 2026.

ELM ELECTRONICS. ELM327: OBD to RS232 interpreter. [S.l.]: Elm Electronics, [s.d.]. Disponível em: https://www.scantool.net/scantool/downloads/103/elm327dsh.pdf. Acesso em: 24 out. 2026.

ESPRESSIF SYSTEMS. ESP32 series datasheet. Shanghai: Espressif Systems, [s.d.]a. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Acesso em: 10 out. 2026.

ESPRESSIF SYSTEMS. ESP-IDF programming guide: Two-Wire Automotive Interface (TWAI). Shanghai: Espressif Systems, [s.d.]b. Disponível em: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/twai.html. Acesso em: 10 out. 2026.

FASTAPI. FastAPI. [S.l.]: [s.n.], [s.d.]. Disponível em: https://fastapi.tiangolo.com/. Acesso em: 24 out. 2026.

FATHY, H. K. et al. Review of hardware-in-the-loop simulation and its prospects in the automotive area. In: MODELING AND SIMULATION FOR MILITARY APPLICATIONS, 2006, Orlando. Proceedings of SPIE, v. 6228, 2006.

FEDERAL TRADE COMMISSION. Nixing the fix: an FTC report to Congress on repair restrictions. Washington, DC: FTC, 2021.

FETTE, I.; MELNIKOV, A. The WebSocket Protocol. RFC 6455. Internet Engineering Task Force, dez. 2011. Disponível em: https://www.rfc-editor.org/info/rfc6455. Acesso em: 24 out. 2026.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. HTTP Semantics. RFC 9110. Internet Engineering Task Force, jun. 2022. Disponível em: https://www.rfc-editor.org/info/rfc9110. Acesso em: 24 out. 2026.

GIL, A. C. Como elaborar projetos de pesquisa. 6. ed. São Paulo: Atlas, 2017.

GUBBI, J. et al. Internet of Things (IoT): a vision, architectural elements, and future directions. Future Generation Computer Systems, v. 29, n. 7, p. 1645-1660, 2013.

GUIMARÃES, A. A. Eletrônica embarcada automotiva. São Paulo: Érica, 2007.

HUNKELER, U.; TRUONG, H. L.; STANFORD-CLARK, A. MQTT-S: a publish/subscribe protocol for wireless sensor networks. In: INTERNATIONAL CONFERENCE ON COMMUNICATION SYSTEMS SOFTWARE AND MIDDLEWARE (COMSWARE), 3., 2008, Bangalore. Proceedings […]. IEEE, 2008. p. 791-798.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO 11898-1: road vehicles: controller area network (CAN): part 1: data link layer and physical signalling. Geneva: ISO, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO 11898-2: road vehicles: controller area network (CAN): part 2: high-speed medium access unit. Geneva: ISO, 2016a.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO 15765-2: road vehicles: diagnostic communication over controller area network (DoCAN): part 2: transport protocol and network layer services. 3. ed. Geneva: ISO, 2016b.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO 15765-4: road vehicles: diagnostic communication over controller area network (DoCAN): part 4: requirements for emissions-related systems. Geneva: ISO, 2021.

JOHANSSON, K. H.; TÖRNGREN, M.; NIELSEN, L. Vehicle applications of controller area network. In: HRISTU-VARSAKELIS, D.; LEVINE, W. S. (ed.). Handbook of networked and embedded control systems. Boston: Birkhäuser, 2005. p. 741-765.

KOSCHER, K. et al. Experimental security analysis of a modern automobile. In: IEEE SYMPOSIUM ON SECURITY AND PRIVACY, 2010, Oakland. Proceedings […]. IEEE, 2010. p. 447-462.

LIGHT, R. A. Mosquitto: server and client implementation of the MQTT protocol. Journal of Open Source Software, v. 2, n. 13, p. 265, 2017.

MALEKIAN, R. et al. Design and implementation of a wireless OBD II fleet management system. IEEE Sensors Journal, v. 17, n. 4, p. 1154-1164, 2017.

NAIK, N. Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP. In: IEEE INTERNATIONAL SYSTEMS ENGINEERING SYMPOSIUM (ISSE), 2017, Vienna. Proceedings […]. IEEE, 2017.

NAVET, N. et al. Trends in automotive communication systems. Proceedings of the IEEE, v. 93, n. 6, p. 1204-1223, 2005.

OASIS. MQTT version 3.1.1. Edited by Andrew Banks and Rahul Gupta. OASIS Standard, 29 Oct. 2014.

PRODANOV, C. C.; FREITAS, E. C. Metodologia do trabalho científico: métodos e técnicas da pesquisa e do trabalho acadêmico. 2. ed. Novo Hamburgo: Feevale, 2013.

PYTHON-CAN. python-can documentation. [S.l.]: [s.n.], [s.d.]. Disponível em: https://python-can.readthedocs.io/. Acesso em: 24 out. 2026.

PYTHON-OBD. python-OBD documentation. [S.l.]: [s.n.], [s.d.]. Disponível em: https://python-obd.readthedocs.io/. Acesso em: 24 out. 2026.

RODRÍGUEZ RODRÍGUEZ, A.; VENTO ÁLVAREZ, J. R.; INOUYE RODRÍGUEZ, R. Implementation of an OBD-II diagnostics tool over CAN-BUS with Arduino. Sistemas & Telemática, v. 16, n. 45, p. 45-53, 2018. DOI: 10.18046/syt.v16i45.2747.

SAE INTERNATIONAL. SAE J1962: diagnostic connector. Warrendale: SAE International, 2016a.

SAE INTERNATIONAL. SAE J1979: E/E diagnostic test modes. Warrendale: SAE International, 2017.

SAE INTERNATIONAL. SAE J2012: diagnostic trouble code definitions. Warrendale: SAE International, 2016b.

SILVA, A. V. da. Desenvolvimento de sistema embarcado para telemetria veicular. 2025. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Controle e Automação) – Universidade Federal de Pernambuco, Recife, 2025. Disponível em: https://repositorio.ufpe.br/bitstream/123456789/67373/1/TCC%20Antonio%20Vitor%20da%20Silva.pdf. Acesso em: 10 out. 2026.

SMITH, C. The car hacker’s handbook: a guide for the penetration tester. San Francisco: No Starch Press, 2016.

SQLITE. About SQLite. [S.l.]: [s.n.], [s.d.]. Disponível em: https://sqlite.org/about.html. Acesso em: 24 out. 2026.

TEXAS INSTRUMENTS. SN65HVD23x 3.3-V CAN bus transceivers: datasheet SLOS346. Dallas: Texas Instruments, [s.d.]. Disponível em: https://www.ti.com/lit/ds/symlink/sn65hvd230.pdf. Acesso em: 10 out. 2026.

UNIÃO EUROPEIA. Regulamento (UE) 2018/858 do Parlamento Europeu e do Conselho, de 30 de maio de 2018. Jornal Oficial da União Europeia, L 151, p. 1- 218, 14 jun. 2018.

ZENG, W.; KHALID, M. A. S.; CHOWDHURY, S. In-vehicle networks outlook: achievements and challenges. IEEE Communications Surveys & Tutorials, v. 18, n. 3, p. 1552-1571, 2016.

APÊNDICE A: ORIGEM DOS NÚMEROS

Cada número reportado neste trabalho tem uma fonte rastreável no repositório. A Tabela 27 relaciona os números principais, o arquivo de origem e a forma de obtê-los. Números não listados aqui são derivações aritméticas simples dos listados (por exemplo, percentuais e razões), ou vêm de tabelas copiadas diretamente do arquivo citado na legenda.

Tabela 27: Origem dos números do Capítulo 5

Número	Valor	Fonte	Como obtido
Testes automatizados	405 aprovados, 2 excluídos	saída de pytest	pytest -q --tb=short --cov=obd em 24/10/2026
Cobertura total	89% (2311 linhas, 254 não cobertas)	saída de pytest-cov	mesmo comando
Cobertura do protocolo	97% a 100%	saída de pytest-cov	mesmo comando
Ciclos e leituras do E1	1907 e 19070	docs/ensaios/e1.json	campos ciclos e leituras
Ciclo do E1 (média, p95)	106,1 ms e 115,6 ms	docs/ensaios/e1.json	campo ciclo_ms
RTT do E1 (média, p95, máx)	10,6; 13,5; 14,1 ms	docs/ensaios/e1.json	campo rtt_ms
Latência do E1 (média, p95, máx)	2,9; 3,5; 42,2 ms	docs/ensaios/e1.json	campo latencia_ms
Erro por PID do E1	Tabela 19	docs/ensaios/e1.json	campo por_pid, subcampo erro
Detecção da P0300	0,79 s	docs/ensaios/e1.json	campo deteccao_falha_s
E2 (ciclo por atraso)	Tabela 21	docs/ensaios/e2.json	campos ciclo_ms e ciclos_por_s de cada execução
E3 (ciclo por PIDs)	Tabela 22	docs/ensaios/e3.json	idem
E4 (perda de quadros)	Tabela 23	docs/ensaios/e4.json	campos timeouts, ciclos, ciclo_ms e recuperacao
E5 (remontagem do VIN)	Tabela 24	docs/ensaios/e5.json	campo remontagem_ms
E6 (19 comandos)	19 de 19	docs/ensaios/e6.json	execução do cliente de soquete cru
E7 (HTTP e MQTT)	Tabela 25	docs/ensaios/ e7_mosquitto.json	campos latencia_ms, ciclos e ciclos_por_s
Máquina de medição	Linux sobre WSL2, Python 3.12.15, 32 núcleos lógicos	campo maquina de cada JSON	registrado pelo executor de ensaios
Linhas de código por módulo	Tabela 7	src/obd/	contagem de linhas dos arquivos de cada pasta, em 24/10/2026
python-OBD 0.7.3 e OBD Auto Doctor	descrição qualitativa	docs/ensaios.md (seção “Cliente real”), docs/duvidas.md (tarefa 09) e observação dos autores	teste test_elm327_python_obd.py e sessão manual

Fonte: elaborada pelos autores.

Observação sobre o E7: o arquivo `docs/ensaios/e7.json` do repositório é o de uma execução sem *broker* real (intermediário em memória) e não é usado nos números deste trabalho. O arquivo `docs/ensaios/e7_mosquitto.json` é o da execução com Mosquitto real, gerada em 24/10/2026.

APÊNDICE B: COMO REPRODUZIR

Instalação e testes

Requisitos: Python 3.12 ou superior.

```
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
make test          # 405 testes
make cov           # com cobertura
make lint          # ruff e mypy
```

Execução da plataforma

```
make run           # servidor, simulador, painel, ELM327 e coletor
# ou, com Docker:
docker compose up app
```

Dashboard em `http://localhost:8000/`, painel de controle em `http://localhost:8001/` e servidor ELM327 em `localhost:35000`. Com `OBD_ELM327_LOG=1`, o servidor imprime cada comando recebido e a resposta enviada.

Execução dos ensaios

```
python -m tests.benchmarks.e1            # 240 s; --duration N muda a duração
python -m tests.benchmarks.e2            # e3, e4, e5, e6 e e7 do mesmo modo
```

Cada ensaio grava, em `runs/<data>/`, um CSV com uma linha por leitura (com `true_value`, `error` e `resolution`), um JSON com configuração, sementes, máquina e métricas, o banco SQLite e os gráficos. Sementes do E1: `seed_model=1` e `seed_ecu=1`; os demais usam as mesmas. Para o E7 com *broker* real, suba o Mosquitto antes (`docker compose --profile mqtt up -d mosquitto`).

Repetição em outra máquina

Para uma comparação entre máquinas, conforme o RNF05: instalar o Python 3.12 ou superior e as dependências, fechar programas que consumam CPU (os tempos são de relógio real), rodar os ensaios E1 a E7 com `--out runs/maquina2`, e comparar os JSON das duas máquinas (campo `maquina`). Valores reais e erros de decodificação dependem só das sementes e devem coincidir; ciclo, RTT e latência dependem do sistema operacional e da CPU, e as diferenças devem ser declaradas. Essa comparação não foi realizada neste trabalho.

Validação com cliente real

```
pytest -q --tb=short tests/integration -m realclient   # exige rede
```

O teste cria um ambiente virtual temporário, instala o python-OBD nele (nunca no ambiente do projeto) e o aponta para o servidor ELM327.

APÊNDICE C: SESSÃO ELM327 COM O SIMULADOR

A Tabela 28 mostra uma sessão real, capturada em 24/10/2026 contra o servidor ELM327 do simulador (`python -m obd run`), por um cliente de soquete cru. Cada linha traz o comando enviado e a resposta, com `\r` representando o retorno de carro. A rotação da consulta `010C` é (12 · 256 + 98)/4 = 792,5 rpm, compatível com a marcha lenta do perfil urbano (800 rpm). O VIN devolvido pelo `0902` é um valor fixo do simulador, sem relação com um veículo real.

Tabela 28: Sessão ELM327 capturada contra o simulador

Comando	Resposta	Comentário
ATZ	ELM327 v1.5	Reinicia e identifica o adaptador
ATE0	ATE0 e OK	Desliga o eco; o próprio comando é ecoado, porque o eco ainda estava ligado
ATL0	OK	Sem avanço de linha
ATH1	OK	Liga cabeçalhos
ATSP0	OK	Protocolo automático
0100	SEARCHING... e 7E8 06 41 00 9C 3A 80 01	Máscara de PIDs suportados (0x01 a 0x20), com cabeçalho
010C	7E8 04 41 0C 0C 62	Rotação: 792,5 rpm
010D	7E8 03 41 0D 00	Velocidade: 0 km/h
0105	7E8 03 41 05 46	Temperatura do arrefecimento: 70 − 40 = 30 °C
ATH0	OK	Desliga cabeçalhos
0902	49 02 01 39 42 ... 31	VIN em uma linha, já remontado
03	NO DATA	Sem códigos de falha armazenados
0600	NO DATA	Modo 06 não suportado
ATDPN	A6	Protocolo automático, ISO 15765-4 CAN 11/500
ATI	ELM327 v1.5	Identificação

Fonte: captura feita pelos autores, em 24/10/2026.

Cada resposta termina com `\r\r>`. Para comparação, a troca observada antes da tarefa 16, com o python-OBD, foi: `ATH1` respondia `OK`, mas as respostas continuavam sem cabeçalho; `ATDPN` e `ATTP6` respondiam `?`; `0600` respondia `7F 06 11`; `0900` respondia `7F 09 12`. A biblioteca descartava os quadros “por serem ímpares” e não lia nenhum PID.

APÊNDICE D: DECISÕES DE PROJETO REGISTRADAS

A especificação deixou pontos em aberto, e cada decisão foi registrada em `docs/duvidas.md`, tarefa por tarefa. A Tabela 29 seleciona as que mais afetam os resultados ou a interpretação deles.

Tabela 29: Decisões de projeto selecionadas

Tarefa	Decisão	Motivo
01	A resolução dos PIDs usa o passo exato (100/255 e 100/128)	Com o valor arredondado, o erro de ida e volta poderia passar de r/2 por arredondamento do critério
02	O ISO-TP usa funções puras e estados com o tempo por parâmetro	Atender à regra de que o protocolo não lê o relógio
02	STmin reservado (0x80 a 0xF0, 0xFA a 0xFF) vale 127 ms	Recomendação da norma
03	Modo 03 sem DTC responde 43 00, não silêncio	O NO DATA é da camada ELM327, não da ECU
03	PID do modo 01 não suportado recebe silêncio	Comportamento da ISO 15765-4
03	O freeze frame é capturado ao armazenar o primeiro DTC não pendente	Fidelidade ao modo 02
04	O ruído entra no valor reportado, não no estado físico limpo	O estado limpo serve de referência para o erro
04	O modelo avança em passos fixos de 10 ms	Resultado independente da granularidade de consulta
04	A perda de quadros descarta a resposta antes do processamento	O coletor vê um timeout, como em um barramento real
05	Recuo progressivo a partir de 3 falhas seguidas	Evitar insistência contra uma ECU muda
05	Ciclo sem leitura válida ainda publica uma amostra vazia	O número de sequência não pode ter lacunas
06	Perdas são as lacunas do número de sequência na execução	Um servidor iniciado no meio da execução não conta o início como perda
06	Uma duplicata responde 202, não é gravada e vira evento	O coletor não deve tratar duplicata como erro
06	A fila de cada cliente WebSocket é ilimitada	Limitar exigiria decidir o que fazer com quem fica para trás; registrado como risco
07	A latência do cabeçalho usa o relógio do navegador	A fonte não estava definida; a mesma máquina evita viés
09	O servidor ELM327 usa o Transport do simulador	ISO-TP, atraso e falhas valem também para o ELM327
10	Os ensaios rodam em um processo, com LoopbackTransport	O valor real só existe no processo da ECU
10	RNF01 e RNF02 são verificados pelo p95	Critério mais rígido que a média
10	Os 500 kbit/s do RNF04 não são medidos	O barramento virtual não tem taxa
12	O MqttPublisher repete a mensagem pendente com recuo	O número de sequência não pode pular nem inverter
16	Serviço ou PID sem suporte, e 7F da ECU, viram NO DATA	Comportamento esperado por clientes como o python-OBD; um chip real pode diferir (dúvida registrada)

Fonte: docs/duvidas.md.